```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

cars = pd.read_csv('/Users/apple/Documents/DA/car_prices.csv')
cars
```

```
     car_ID  symboling                   CarName fueltype
aspiration  \
0         1          3        alfa-romero giulia      gas        std

1         2          3       alfa-romero stelvio      gas        std

2         3          1  alfa-romero Quadrifoglio      gas        std

3         4          2               audi 100 ls      gas        std

4         5          2                audi 100ls      gas        std

..      ...        ...                       ...      ...        ...

200     201         -1           volvo 145e (sw)      gas        std

201     202         -1               volvo 144ea      gas      turbo

202     203         -1               volvo 244dl      gas        std

203     204         -1                 volvo 246   diesel      turbo

204     205         -1               volvo 264gl      gas      turbo


    doornumber      carbody drivewheel enginelocation  wheelbase  ...
\
0          two  convertible        rwd          front       88.6  ...

1          two  convertible        rwd          front       88.6  ...

2          two    hatchback        rwd          front       94.5  ...

3         four        sedan        fwd          front       99.8  ...

4         four        sedan        4wd          front       99.4  ...

..         ...          ...        ...            ...        ...  ...

200       four        sedan        rwd          front      109.1  ...

201       four        sedan        rwd          front      109.1  ...

202       four        sedan        rwd          front      109.1  ...
```

```
203       four        sedan        rwd          front      109.1  ...

204       four        sedan        rwd          front      109.1  ...


     enginesize  fuelsystem  boreratio  stroke compressionratio
horsepower  \
0           130        mpfi       3.47    2.68              9.0
111
1           130        mpfi       3.47    2.68              9.0
111
2           152        mpfi       2.68    3.47              9.0
154
3           109        mpfi       3.19    3.40             10.0
102
4           136        mpfi       3.19    3.40              8.0
115
..          ...         ...        ...     ...              ...
...
200         141        mpfi       3.78    3.15              9.5
114
201         141        mpfi       3.78    3.15              8.7
160
202         173        mpfi       3.58    2.87              8.8
134
203         145         idi       3.01    3.40             23.0
106
204         141        mpfi       3.78    3.15              9.5
114

     peakrpm citympg  highwaympg    price
0       5000      21          27  13495.0
1       5000      21          27  16500.0
2       5000      19          26  16500.0
3       5500      24          30  13950.0
4       5500      18          22  17450.0
..       ...     ...         ...      ...
200     5400      23          28  16845.0
201     5300      19          25  19045.0
202     5500      18          23  21485.0
203     4800      26          27  22470.0
204     5400      19          25  22625.0

[205 rows x 26 columns]
```

```
cars.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 205 entries, 0 to 204
```

```
Data columns (total 26 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   car_ID            205 non-null    int64
 1   symboling         205 non-null    int64
 2   CarName           205 non-null    object
 3   fueltype          205 non-null    object
 4   aspiration        205 non-null    object
 5   doornumber        205 non-null    object
 6   carbody           205 non-null    object
 7   drivewheel        205 non-null    object
 8   enginelocation    205 non-null    object
 9   wheelbase         205 non-null    float64
 10  carlength         205 non-null    float64
 11  carwidth          205 non-null    float64
 12  carheight         205 non-null    float64
 13  curbweight        205 non-null    int64
 14  enginetype        205 non-null    object
 15  cylindernumber    205 non-null    object
 16  enginesize        205 non-null    int64
 17  fuelsystem        205 non-null    object
 18  boreratio         205 non-null    float64
 19  stroke            205 non-null    float64
 20  compressionratio  205 non-null    float64
 21  horsepower        205 non-null    int64
 22  peakrpm           205 non-null    int64
 23  citympg           205 non-null    int64
 24  highwaympg        205 non-null    int64
 25  price             205 non-null    float64
dtypes: float64(8), int64(8), object(10)
memory usage: 41.8+ KB
```

```
cars.describe()
```

```
           car_ID   symboling   wheelbase   carlength    carwidth
carheight  \
count  205.000000  205.000000  205.000000  205.000000  205.000000
205.000000
mean   103.000000    0.834146   98.756585  174.049268   65.907805
53.724878
std     59.322565    1.245307    6.021776   12.337289    2.145204
2.443522
min      1.000000   -2.000000   86.600000  141.100000   60.300000
47.800000
25%     52.000000    0.000000   94.500000  166.300000   64.100000
52.000000
50%    103.000000    1.000000   97.000000  173.200000   65.500000
54.100000
75%    154.000000    2.000000  102.400000  183.100000   66.900000
55.500000
```

```
max    205.000000    3.000000  120.900000  208.100000   72.300000
59.800000

        curbweight  enginesize   boreratio      stroke
compressionratio  \
count   205.000000  205.000000  205.000000  205.000000
205.000000
mean   2555.565854  126.907317    3.329756    3.255415
10.142537
std     520.680204   41.642693    0.270844    0.313597
3.972040
min    1488.000000   61.000000    2.540000    2.070000
7.000000
25%    2145.000000   97.000000    3.150000    3.110000
8.600000
50%    2414.000000  120.000000    3.310000    3.290000
9.000000
75%    2935.000000  141.000000    3.580000    3.410000
9.400000
max    4066.000000  326.000000    3.940000    4.170000
23.000000

       horsepower      peakrpm     citympg  highwaympg         price
count  205.000000   205.000000  205.000000  205.000000    205.000000
mean   104.117073  5125.121951   25.219512   30.751220  13276.710571
std     39.544167   476.985643    6.542142    6.886443   7988.852332
min     48.000000  4150.000000   13.000000   16.000000   5118.000000
25%     70.000000  4800.000000   19.000000   25.000000   7788.000000
50%     95.000000  5200.000000   24.000000   30.000000  10295.000000
75%    116.000000  5500.000000   30.000000   34.000000  16503.000000
max    288.000000  6600.000000   49.000000   54.000000  45400.000000
```

I. TÌM HIỂU DỮ LIỆU VÀ TIỀN XỬ LÍ DỮ LIỆU

1. Tìm hiểu dữ liệu

```
- Bộ dữ liệu gồm 205 bản ghi.
- Bộ dữ liệu gồm 26 cột, trong đó:
+ 8 cột có kiểu dữ liệu float
+ 8 cột có kiểu dữ liệu int
+ 10 cột có kiểu dữ liệu object
- Bộ dữ liệu không có giá trị khuyết thiếu ở tất cả các dòng và các
cột.
```

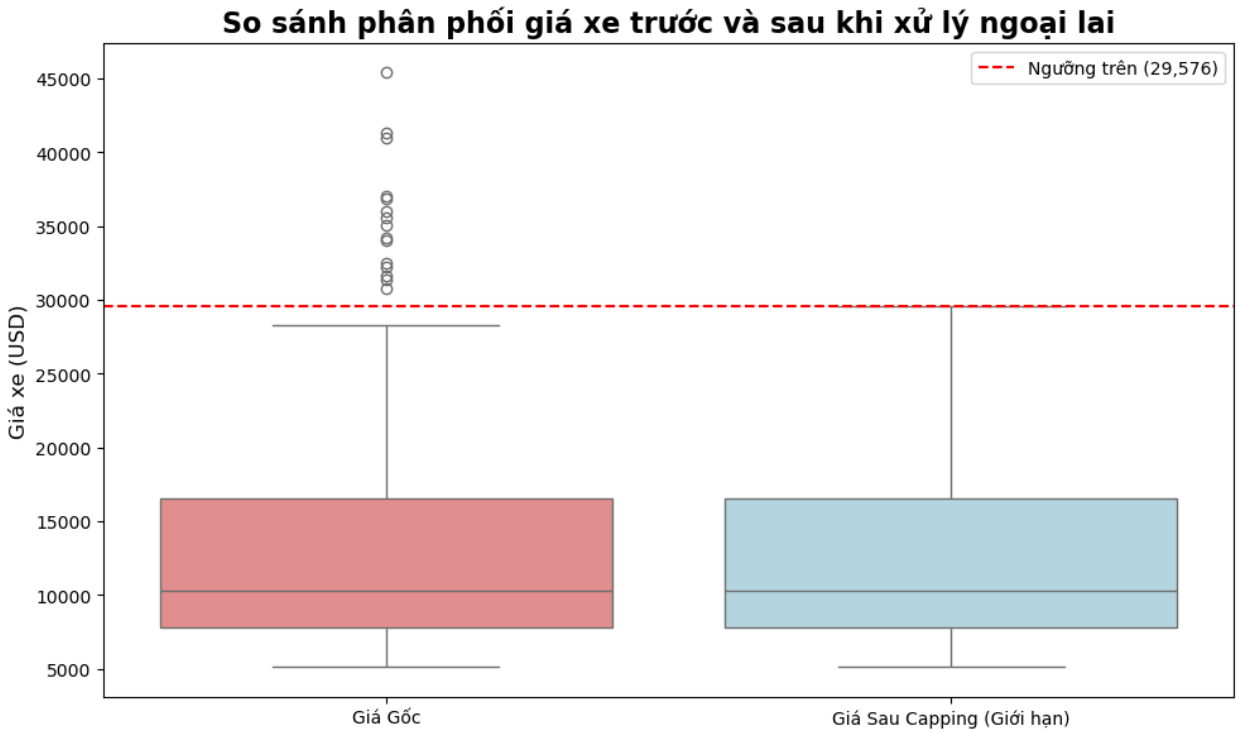
2. Làm sạch dữ liệu

```
# Trích xuất tên hãng thành 1 cột riêng
cars['Manufacturer'] = cars['CarName'].apply(lambda x: x.split(' ')
[0])
```

```
# Chuẩn hoá tên hãng sản xuất (xử lí lỗi chính tả và tên viết tắt)
cars['Manufacturer'] = cars['Manufacturer'].str.lower()
cars.replace({'Manufacturer':{
    'maxda': 'mazda',
    'toyouta': 'toyota',
    'vokswagen': 'volkswagen',
    'vw': 'volkswagen',
    'porcshce': 'porsche'
}}, inplace=True)
```

3. Xử lí dữ liệu ngoại lai

```
# Xử lý Ngoại lai (Capping: Thay thế các giá trị ngoại lai bằng Ngưỡng)
Q1 = cars['price'].quantile(0.25)
Q3 = cars['price'].quantile(0.75)
IQR = Q3 - Q1
Upper_Bound = Q3 + 1.5 * IQR
cars['price_capped'] = np.where(cars['price'] > Upper_Bound,
Upper_Bound, cars['price'])

# Chuẩn bị dữ liệu cho biểu đồ Box Plot
# Tạo DataFrame để chồng dữ liệu lên nhau
plot_cars = pd.DataFrame({
    'Giá Trị': cars['price'],
    'Loại': 'Giá Gốc'
})
plot_cars = pd.concat([
    plot_cars,
    pd.DataFrame({
        'Giá Trị': cars['price_capped'],
        'Loại': 'Giá Sau Capping (Giới hạn)'
    })
])

# Vẽ biểu đồ Box Plot So sánh ---
plt.figure(figsize=(10, 6))
sns.boxplot(x='Loại', y='Giá Trị', hue='Loại', data=plot_cars,
palette=['lightcoral', 'lightblue'], legend=False)
plt.axhline(y=Upper_Bound, color='r', linestyle='--', label=f'Ngưỡng
trên ({Upper_Bound:,.0f})')
plt.title('So sánh phân phối giá xe trước và sau khi xử lý ngoại lai',
fontsize=16, fontweight='bold')
plt.ylabel('Giá xe (USD)', fontsize=12)
plt.xlabel('')
plt.legend()
plt.tight_layout()
```




```
- Giá Gôć: Phân phôí có nhiêù điêḿ ngoại lai (châḿ đen) trải dài lên
đêń giá trị $45,000 USD. Những điêḿ này nằm ngoài hộp và râu, kéo dài
phương sai và làm lệch mô hình hôì quy.
- Giá Sau Capping:
+ Không còn ngoại lai: Tât́ cả các châḿ đen đã biêń mât́.
+ Giới hạn Max: Giá trị tôí đa của phân phôí này hiện đã được giới hạn
ở Ngưỡng trên, được biêủ thị bằng đường đứt nét màu đỏ.
=> Việc xử̉lí ngoại lai (capping) đã loại bỏ các giá trị cực đoan,
giúp phân phôí giá trở nên chuâń hơn và ôń định hơn cho mô hình hôì
quy.

# Xử̉ lí ngoại lai cho các cột sô´

numerical_cols =
cars.select_dtypes(include=np.number).columns.tolist()
# Loại bỏ car_ID, symboling và price
numerical_cols = [col for col in numerical_cols if col not in
['car_ID', 'symboling', 'price']]

#capped_columns = []
for col in numerical_cols:
    Q1 = cars[col].quantile(0.25)
    Q3 = cars[col].quantile(0.75)
    IQR = Q3 - Q1
    upper_bound = Q3 + 1.5 * IQR

    # Giới hạn giá trị
```

```
    new_col = f'{col}_capped'
    cars[new_col] = np.where(cars[col] > upper_bound, upper_bound,
cars[col])
    #capped_columns.append(new_col_name)
```

4. Chuẩn hoá và mã hoá dữ liệu

```
# Xem xét, đánh giá các biê´n sô´

# Biê´n phân loại mã hoá One-Hot Encoding
+ drivewheel
+ enginelocation
+ fueltype
+ carbody

# Biê´n liên tục chuẩ̉n hóa Z-Score
+ horsepower
+ enginesize
+ curbweight
+ carlength
+ carwidth

# Mã hoá
categorical_cols = ['drivewheel', 'enginelocation', 'fueltype',
'carbody']
cars = pd.get_dummies(cars, columns=categorical_cols)

# Chuẩ̉n hoá
from sklearn.preprocessing import StandardScaler

standard_features = ['horsepower_capped', 'enginesize_capped',
'curbweight_capped', 'carlength_capped', 'carwidth_capped']
scaler = StandardScaler()
cars_s = scaler.fit_transform(cars[standard_features])
cars_s = pd.DataFrame(cars_s, columns=standard_features)
print(cars_s)
```

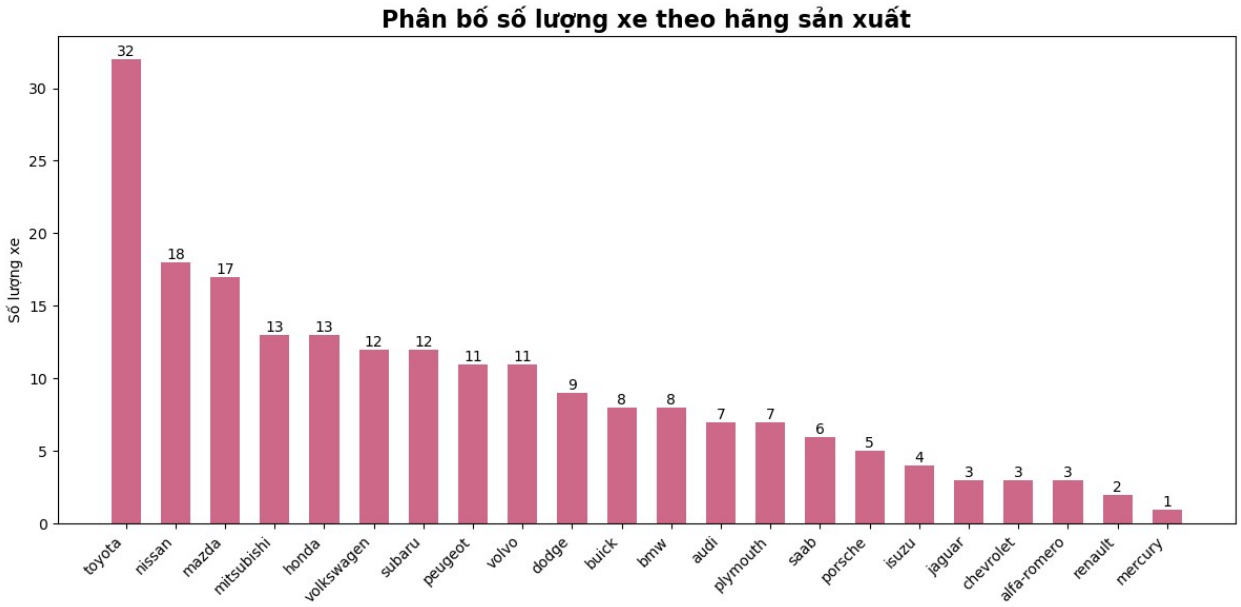
```
     horsepower_capped  enginesize_capped  curbweight_capped  \
0             0.229801           0.160196          -0.014566
1             0.229801           0.160196          -0.014566
2             1.441341           0.809329           0.514882
3            -0.023777          -0.459430          -0.420797
4             0.342502           0.337232           0.516807
..                 ...                ...                ...
200           0.314327           0.484762           0.763241
201           1.610393           0.484762           0.949992
202           0.877834           1.428955           0.878757
203           0.088924           0.602787           1.273437
204           0.314327           0.484762           0.975021
```

```
     carlength_capped  carwidth_capped
0           -0.426521        -0.858695
1           -0.426521        -0.858695
2           -0.231513        -0.184978
3            0.207256         0.151880
4            0.207256         0.248125
..                ...              ...
200          1.198549         1.451189
201          1.198549         1.403066
202          1.198549         1.451189
203          1.198549         1.451189
204          1.198549         1.451189

[205 rows x 5 columns]

horsepower_capped của chiếc xe đầu tiên là 0.229801, có nghĩa là mã
lực của nó cao hơn 0.229801 độ lệch chuẩn so với công suất trung bình
của
tập dữ liệu.
```

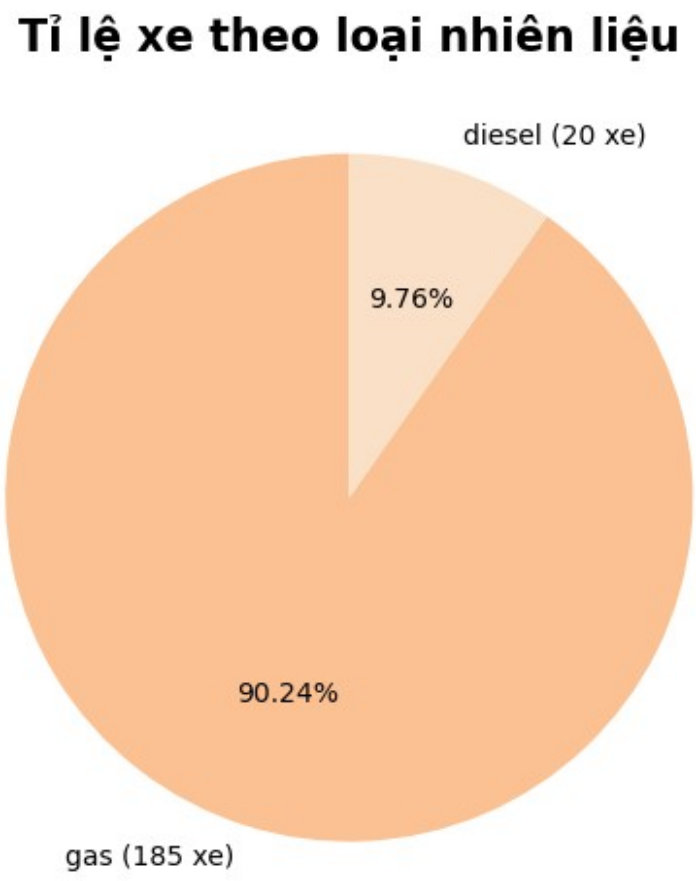
II. TRỰC QUAN HOÁ DỮ LIỆU, PHÂN TÍCH CÁC ĐẶC TRƯNG

1. Biểu đồ mô tả dữ liệu

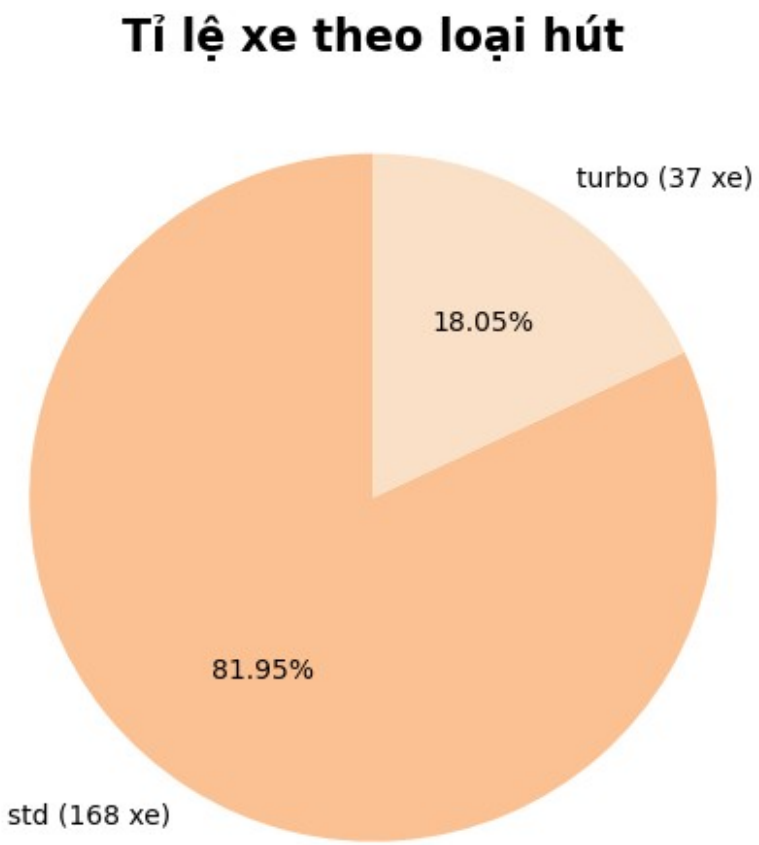
```
manufacturer = cars['Manufacturer'].value_counts()
x = manufacturer.index
y = manufacturer.values

fig, ax = plt.subplots(figsize=(12,6))
bars = ax.bar(x,y, width=0.6, color='#CD6889')
ax.bar_label(bars, label_type='edge')
ax.set_title('Phân bố số lượng xe theo hãng sản xuất', fontsize=16,
fontweight='bold')
ax.set_xticks(x)
ax.set_xticklabels(x, rotation=45, ha='right')
ax.set_ylabel('Số lượng xe')

plt.tight_layout()
plt.show()
```



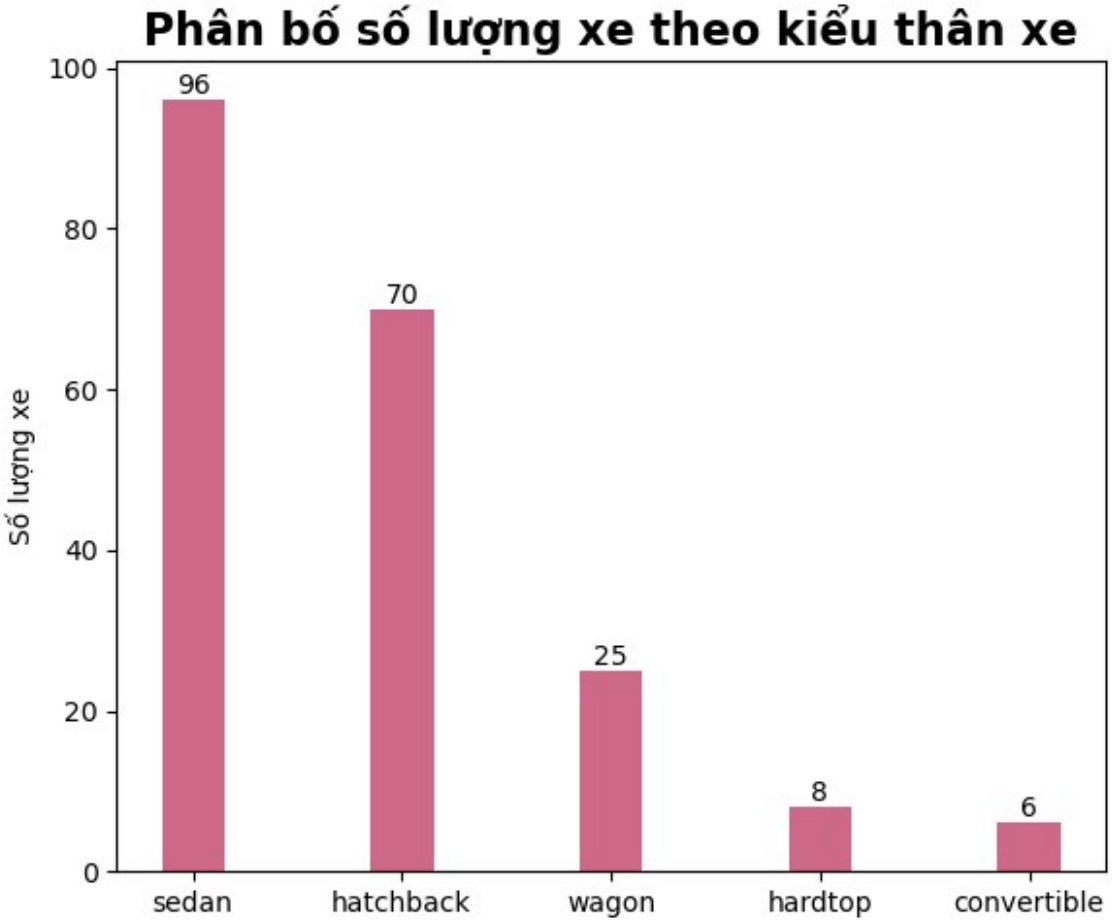

```
fuel_type = cars['fueltype'].value_counts()
ratio_fuel_type = cars['fueltype'].value_counts(normalize=True)*100
labels = [
    f"{fuel} ({count} xe)"
    for fuel, count in zip(fuel_type.index, fuel_type.values)
]
sizes = ratio_fuel_type.values
colors = ['#FBC193', '#FAE0C7']

fig, ax = plt.subplots(figsize=(5,5))
ax.pie(sizes, labels=labels, colors=colors, autopct='%1.2f%%',
startangle=90)
ax.set_title('Tỉ lệ xe theo loại nhiên liệu', fontsize=16,
fontweight='bold')

plt.tight_layout()
plt.show()
```



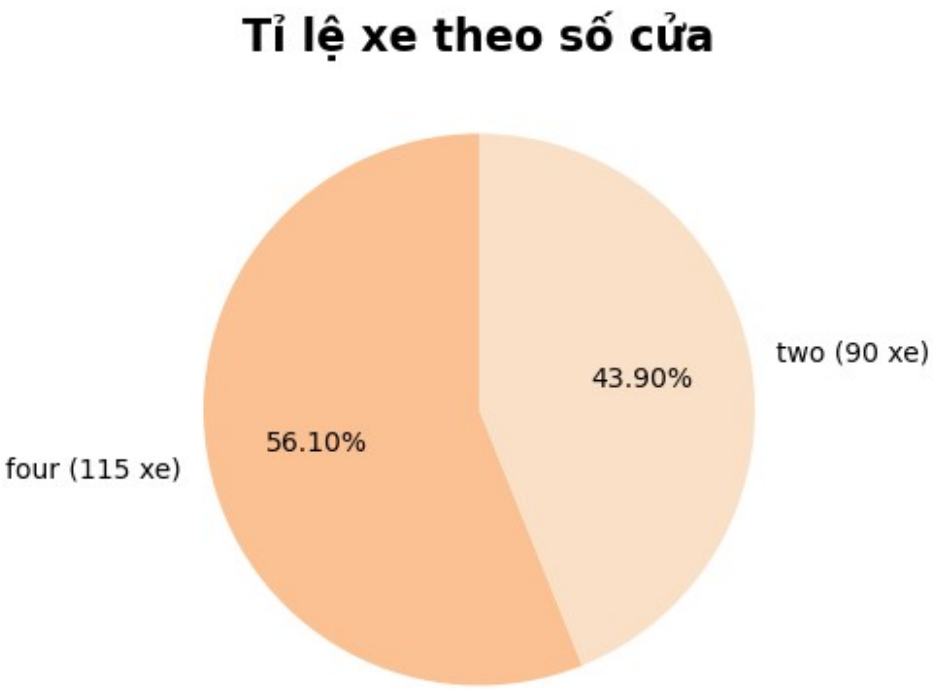

```
aspiration = cars['aspiration'].value_counts()
ratio_aspiration = cars['aspiration'].value_counts(normalize=True)*100
labels = [
    f"{aspiration} ({count} xe)"
    for aspiration, count in zip(aspiration.index, aspiration.values)
]
sizes = ratio_aspiration.values
colors = ['#FBC193', '#FAE0C7']

fig, ax = plt.subplots(figsize=(5,5))
ax.pie(sizes, labels=labels, colors=colors, autopct='%1.2f%%',
startangle=90)
ax.set_title('Tỉ lệ xe theo loại hút', fontsize=16, fontweight='bold')

plt.tight_layout()
plt.show()
```



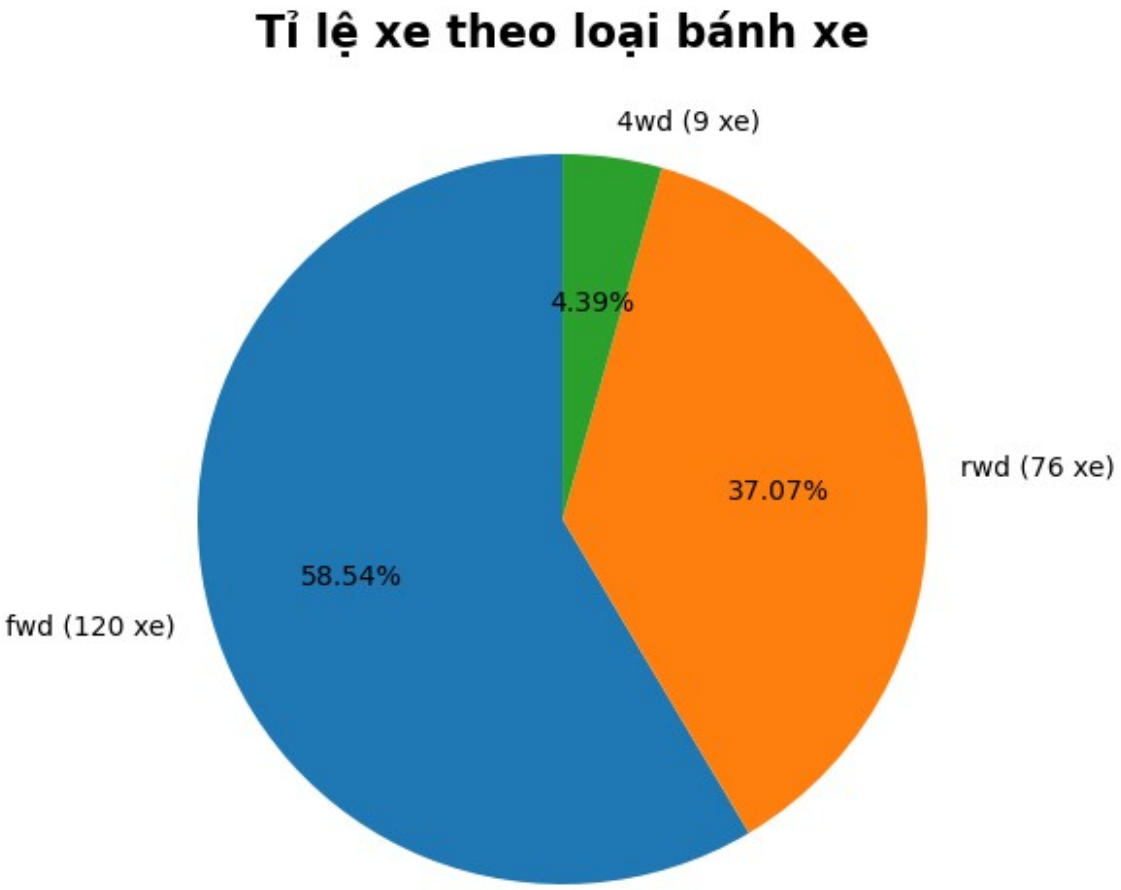

```
carbody = cars['carbody'].value_counts()
x = carbody.index
y = carbody.values

fig, ax = plt.subplots(figsize=(6,5))
bars = ax.bar(x,y, width=0.3, color='#CD6889')
ax.bar_label(bars, label_type='edge')
ax.set_title('Phân bô´sô´lượng xe theo kiêủ thân xe', fontsize=16,
fontweight='bold')
ax.set_xticks(x)
ax.set_xticklabels(x)
ax.set_ylabel('Sô´lượng xe')

plt.tight_layout()
plt.show()
```




```
doornumber = cars['doornumber'].value_counts()
ratio_doornumber = cars['doornumber'].value_counts(normalize=True)*100
labels = [
    f"{doornumber} ({count} xe)"
    for doornumber, count in zip(doornumber.index, doornumber.values)
]
sizes = ratio_doornumber.values
colors = ['#FBC193', '#FAE0C7']

fig, ax = plt.subplots(figsize=(5,5))
ax.pie(sizes, labels=labels, colors=colors, autopct='%1.2f%%',
startangle=90)
ax.set_title('Tỉ lệ xe theo sô´cưả', fontsize=16, fontweight='bold')

plt.tight_layout()
plt.show()
```




```
drivewheel = cars['drivewheel'].value_counts()
ratio_drivewheel = cars['drivewheel'].value_counts(normalize=True)*100
labels = [
    f"{drivewheel} ({count} xe)"
    for drivewheel, count in zip(drivewheel.index, drivewheel.values)
]
sizes = ratio_drivewheel.values
#colors = ['#FBC193', '#FAE0C7']

fig, ax = plt.subplots(figsize=(6,6))
ax.pie(sizes, labels=labels, autopct='%1.2f%%', startangle=90)
ax.set_title('Tỉ lệ xe theo loại bánh xe', fontsize=16,
fontweight='bold')

plt.tight_layout()
plt.show()
```

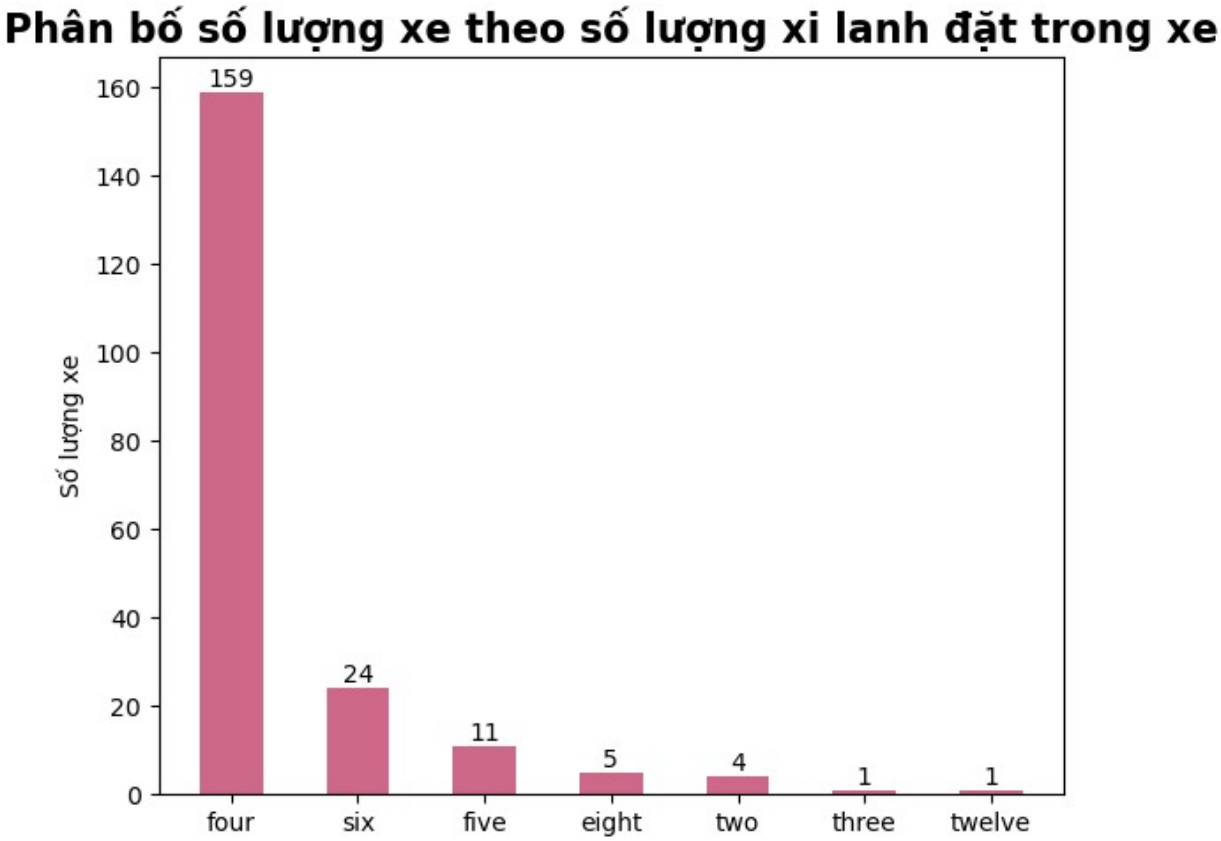



```
cars.head()

   car_ID  symboling                   CarName aspiration
doornumber  \
0       1          3        alfa-romero giulia        std        two

1       2          3       alfa-romero stelvio        std        two

2       3          1  alfa-romero Quadrifoglio        std        two

3       4          2               audi 100 ls        std       four

4       5          2                audi 100ls        std       four


   wheelbase  carlength  carwidth  carheight  curbweight  ...
drivewheel_rwd  \
0       88.6      168.8      64.1       48.8        2548  ...
True
1       88.6      168.8      64.1       48.8        2548  ...
```

```
True
2       94.5      171.2      65.5       52.4        2823  ...
True
3       99.8      176.6      66.2       54.3        2337  ...
False
4       99.4      176.6      66.4       54.3        2824  ...
False

  enginelocation_front  enginelocation_rear fueltype_diesel
fueltype_gas  \
0                 True                False           False
True
1                 True                False           False
True
2                 True                False           False
True
3                 True                False           False
True
4                 True                False           False
True

   carbody_convertible  carbody_hardtop  carbody_hatchback
carbody_sedan  \
0                 True            False              False
False
1                 True            False              False
False
2                False            False               True
False
3                False            False              False
True
4                False            False              False
True

   carbody_wagon
0          False
1          False
2          False
3          False
4          False

[5 rows x 50 columns]
```

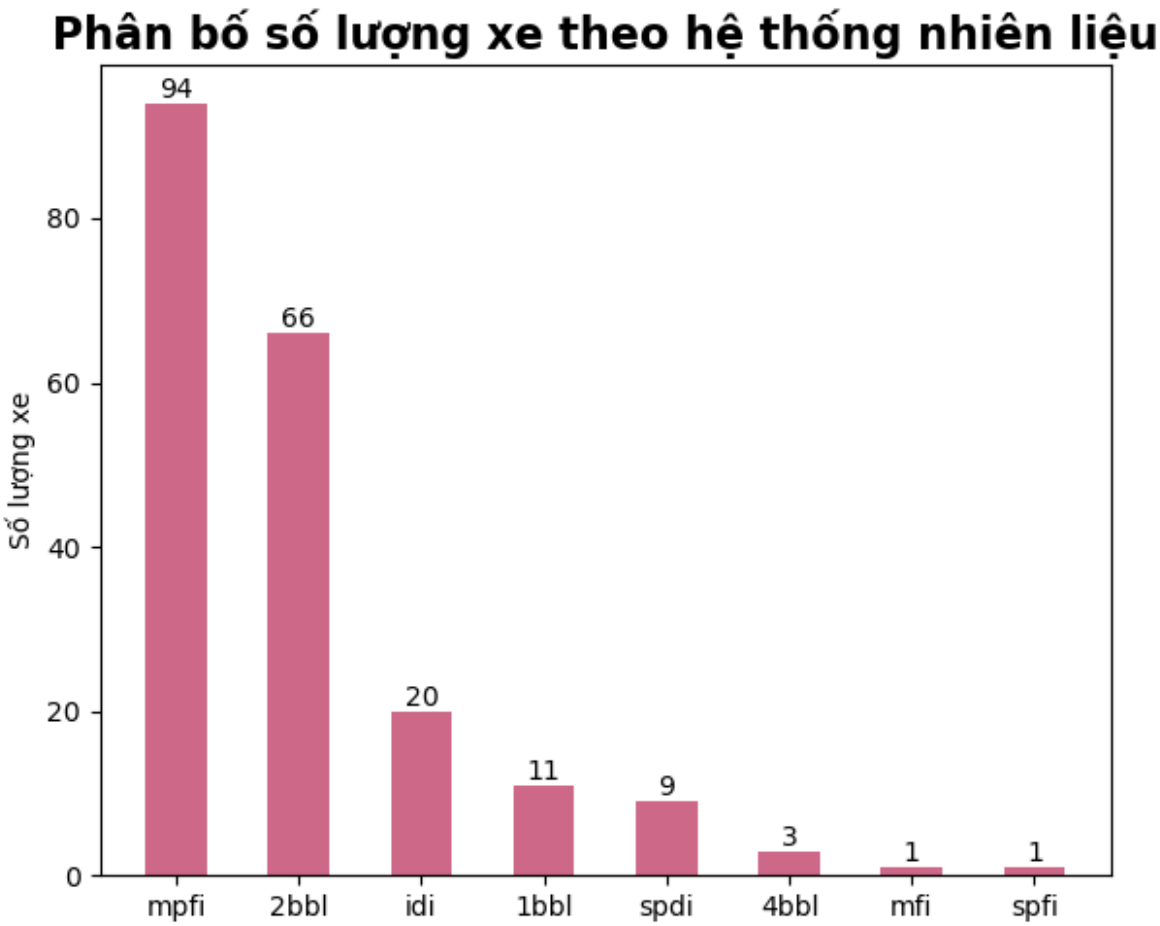
```
cylindernumber = cars['cylindernumber'].value_counts()
x = cylindernumber.index
y = cylindernumber.values

fig, ax = plt.subplots(figsize=(6,5))
bars = ax.bar(x,y, width=0.5, color='#CD6889')
ax.bar_label(bars, label_type='edge')
```

```
ax.set_title('Phân bô´sô´lượng xe theo sô´lượng xi lanh đặt trong xe',
fontsize=16, fontweight='bold')
ax.set_xticks(x)
ax.set_xticklabels(x)
ax.set_ylabel('Sô´lượng xe')

plt.tight_layout()
plt.show()
```



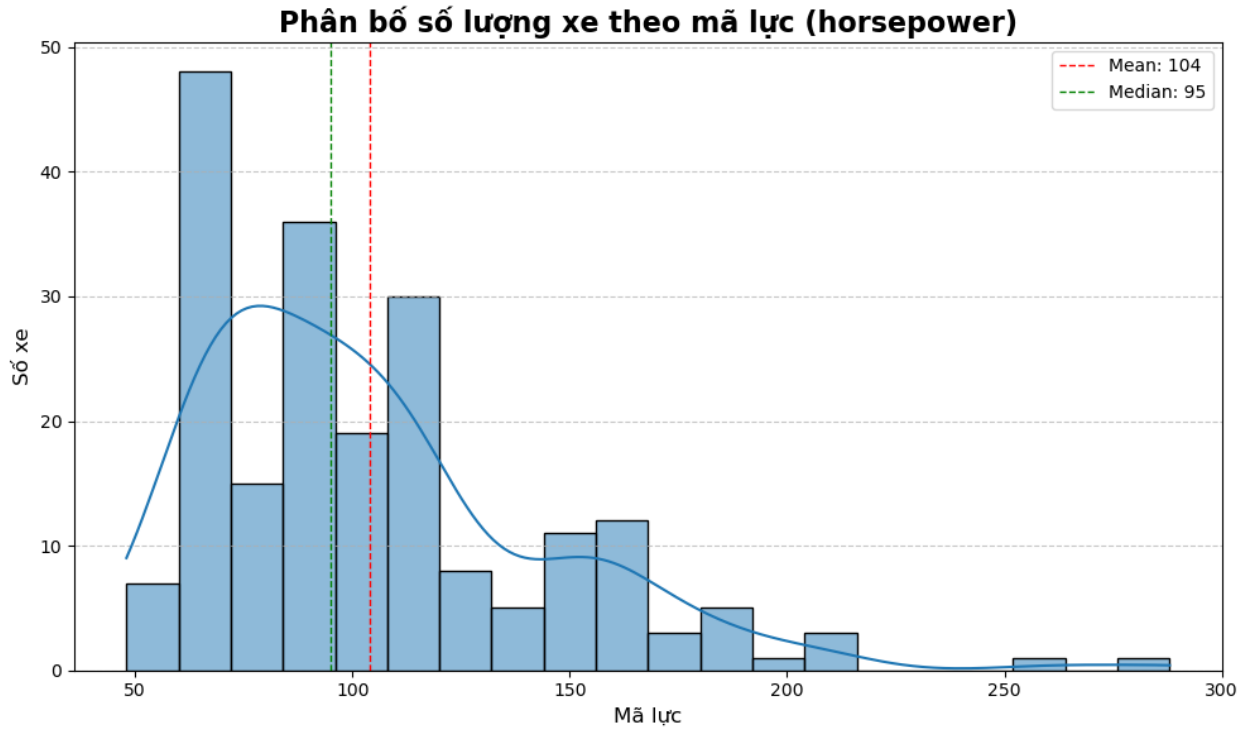

```
fuelsystem = cars['fuelsystem'].value_counts()
x = fuelsystem.index
y = fuelsystem.values

fig, ax = plt.subplots(figsize=(6,5))
bars = ax.bar(x,y, width=0.5, color='#CD6889')
ax.bar_label(bars, label_type='edge')
ax.set_title('Phân bô´sô´lượng xe theo hệ thông nhiên liệu',
fontsize=16, fontweight='bold')
ax.set_xticks(x)
ax.set_xticklabels(x)
ax.set_ylabel('Sô´lượng xe')
```

```
plt.tight_layout()
plt.show()
```



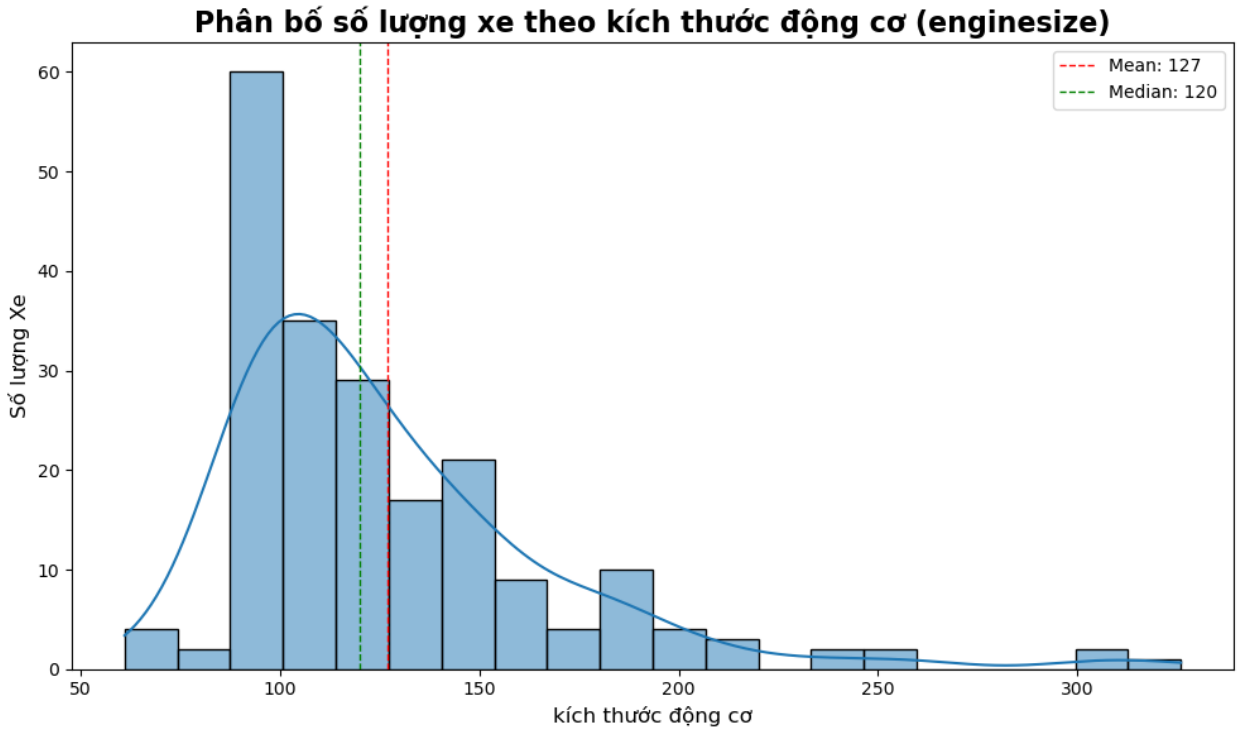

```
fig, ax = plt.subplots(figsize=(10, 6))
sns.histplot(cars['horsepower'], bins=20, kde=True)

mean_enginesize = cars['horsepower'].mean()
median_enginesize = cars['horsepower'].median()

plt.axvline(mean_enginesize, color='red', linestyle='dashed',
linewidth=1, label=f'Mean: {mean_enginesize:.0f}')
plt.axvline(median_enginesize, color='green', linestyle='dashed',
linewidth=1, label=f'Median: {median_enginesize:.0f}')

plt.title('Phân bô´sô´lượng xe theo mã lực (horsepower)', fontsize=16,
fontweight='bold')
plt.xlabel('Mã lực', fontsize=12)
plt.ylabel('Sô´lượng xe', fontsize=12)
plt.grid(axis='y', linestyle='--', alpha=0.7)
```

```
plt.legend()
plt.tight_layout()
```



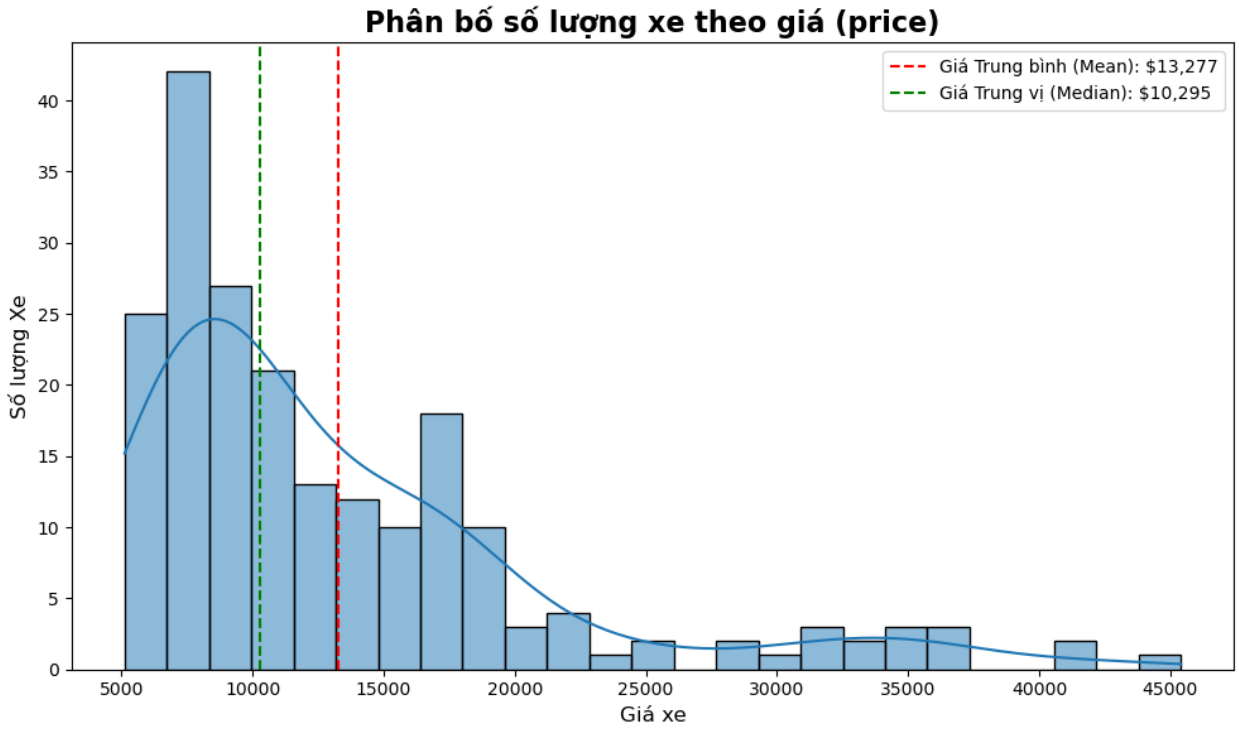

```
- Phân phôí Lệch: Phân phôí bị lệch phải, có nghĩa là phâǹ lớn các xe
có mã lực thâp hơn.
- Dải mã lực phổbiếń nhâť: Hâù hêť các xe nằm trong dải mã lực từ
khoảng 60 đêń 120 mã lực. Đây là nơi tâǹ suâť cao nhâť.
- Giá trị Ngoại lai/Công suâť Cao: Có một sốlượng nhỏ xe có mã lực
râť cao, kéo dài phân phôí vềphía bên phải
(ví dụ: trên 200 mã lực).

plt.figure(figsize=(10, 6))
sns.histplot(cars['enginesize'], kde=True, bins=20)

mean_enginesize = cars['enginesize'].mean()
median_enginesize = cars['enginesize'].median()

plt.axvline(mean_enginesize, color='red', linestyle='dashed',
linewidth=1, label=f'Mean: {mean_enginesize:.0f}')
plt.axvline(median_enginesize, color='green', linestyle='dashed',
linewidth=1, label=f'Median: {median_enginesize:.0f}')

plt.title('Phân bốsốlượng xe theo kích thước động cơ (enginesize)',
fontsize=16, fontweight='bold')
plt.xlabel('kích thước động cơ', fontsize=12)
plt.ylabel('Sốlượng Xe', fontsize=12)
```

```
plt.legend()
plt.tight_layout()
```




```
- Phân phối Lệch: Biểu đồ hiển thị sự phân phối lệch phải. Điều này có
nghĩa là phần lớn các xe có kích thước động cơ nhỏ hơn hoặc trung
bình,
và chỉ một số ít xe có động cơ rất lớn.
- Giá trị Trung bình và Trung vị:
+ Đường Trung vị (Median - màu xanh lá) nằm ở 120.
+ Đường Trung bình (Mean - màu đỏ) nằm ở 127 (cao hơn Trung vị), điều
này xác nhận phân phối bị kéo về phía các giá trị lớn hơn.
=> Biểu đồ này giúp xác định rằng động cơ nhỏ đến trung bình là phổ
biến nhất trong thị trường xe này.

plt.figure(figsize=(10, 6))
sns.histplot(cars['price'], kde=True, bins=25)

mean_price = cars['price'].mean()
median_price = cars['price'].median()

plt.axvline(mean_price, color='red', linestyle='dashed',
linewidth=1.5, label=f'Giá Trung bình (Mean): ${mean_price:,.0f}')
plt.axvline(median_price, color='green', linestyle='dashed',
linewidth=1.5, label=f'Giá Trung vị (Median): ${median_price:,.0f}')

plt.title('Phân bố số lượng xe theo giá (price)', fontsize=16,
fontweight='bold')
```

```
plt.xlabel('Giá xe', fontsize=12)
plt.ylabel('Sô´lượng Xe', fontsize=12)
plt.legend()
plt.tight_layout()
```

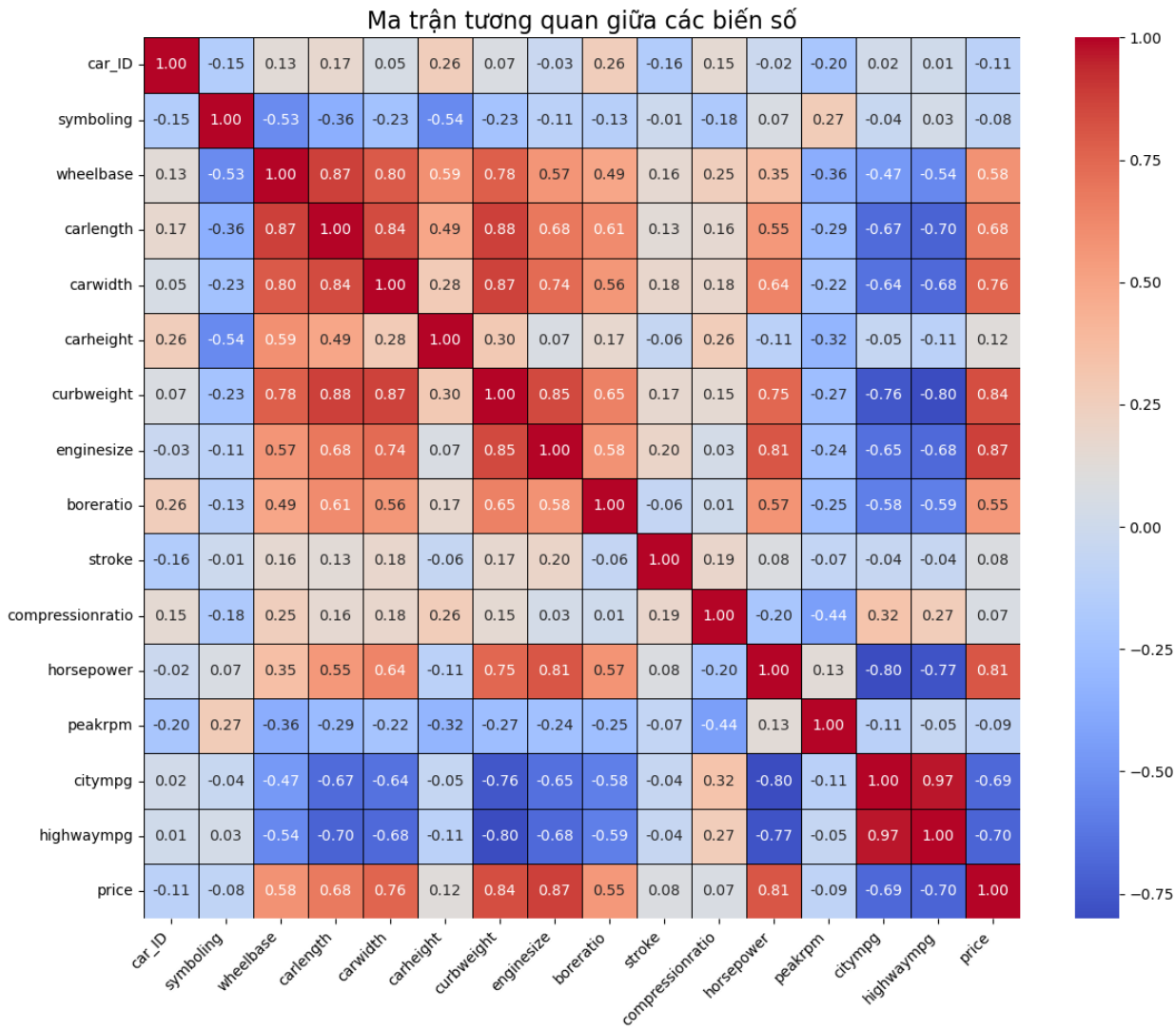



```
- Phân phôí lệch phải mạnh cho thấý phâǹ lớn các mâũ xe có giá thâṕ,
trong khi chỉ một sô´lượng nhỏ các mâũ xe nằm ở phân khúc giá cao (cao

câṕ/sang trọng).
- Phân khúc giá phổbiêń nhât́:
+ Phâǹ lớn các xe (tâǹ suât́ cao nhât́) có giá nằm trong khoảng từ
khoảng $5,000 đêń $10,000 USD.
+ Sau mức giá $15,000, sô´lượng xe giảm mạnh.
- Giá trị trung bình và trung vị:
+ Giá trung vị (Median - màu xanh lá): $10,295
+ Giá trung bình (Mean - màu đỏ): $13,277
+ Việc giá trung bình cao hơn đáng kểgiá trung vị là một dâú hiệu rõ
ràng của sự lệch phải mạnh, do các xe đăt́ tiêǹ (giá ngoại lai) đã kéo
giá trị trung bình lên cao.
```

2. Phân tích tương quan

```
numerical_cols = cars.select_dtypes(include=[np.number])
corr_matrix = numerical_cols.corr()

plt.figure(figsize=(12, 10))
```

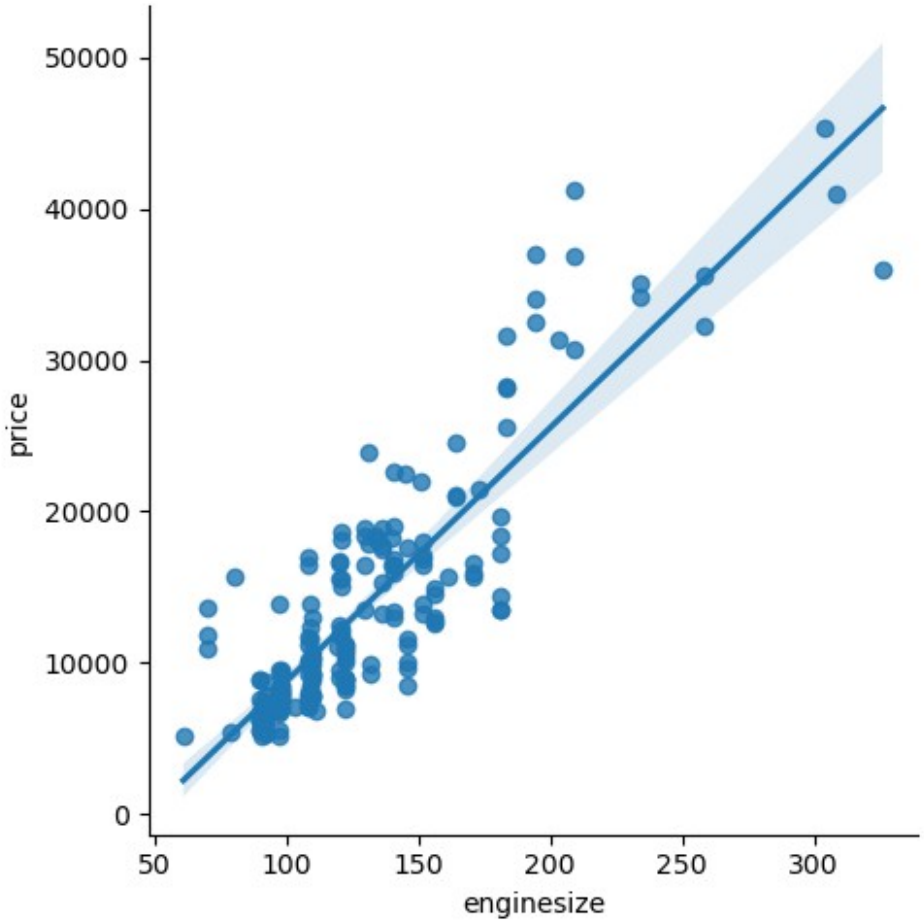
```
sns.heatmap(
    corr_matrix,
    annot=True,       # Hiển thị giá trị tương quan trên ô
    fmt=".2f",        # Định dạng 2 chữ số thập phân
    cmap='coolwarm', # Bảng màu
    linewidths=.5,    # Độ rộng đường kẻ giữa các ô
    linecolor='black'
)

plt.title('Ma trận tương quan giữa các biến số', fontsize=16)
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()
```




```
- Các biến có môi quan hệ tương quan dương với biến mục tiêu (price): cả 2 cùng tăng
```

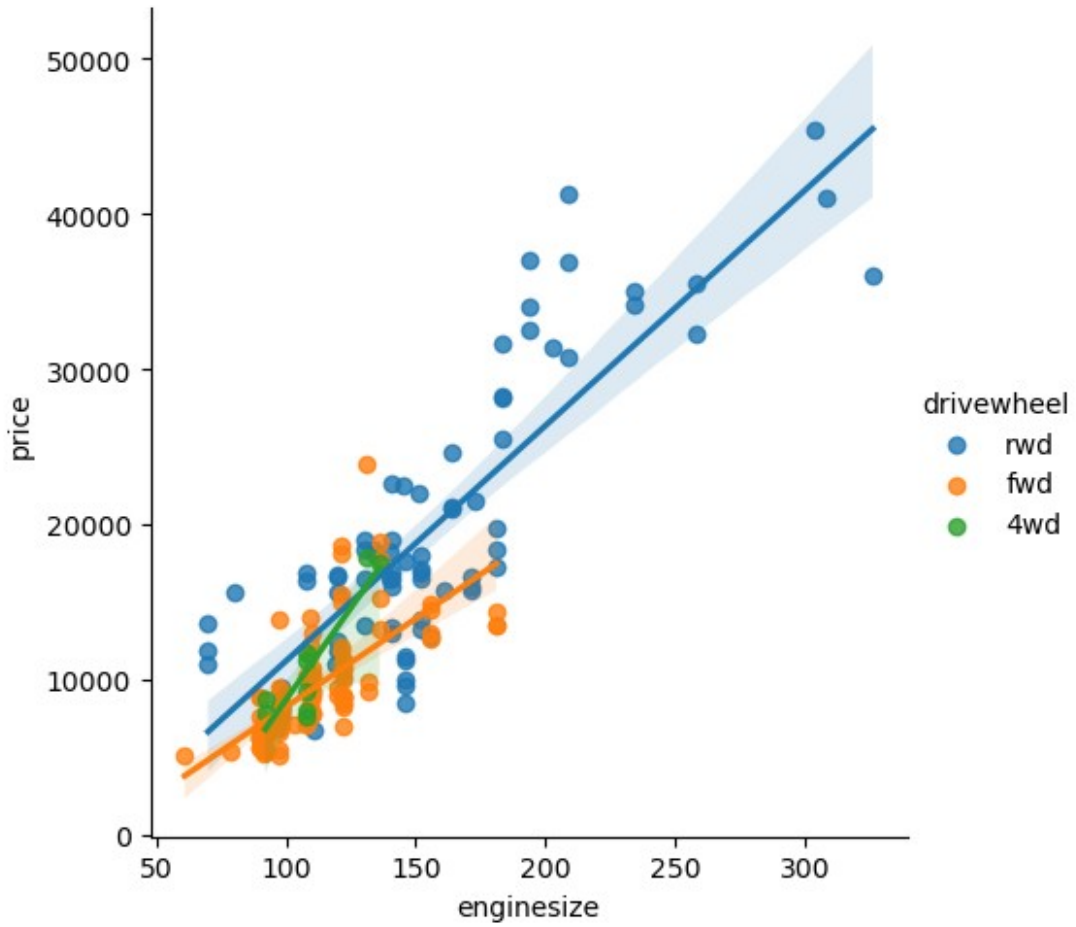
```
+ wheelbase
+ carlength
+ carwidth
+ curbweight
+ enginesize
+ boreratio
+ horsepower
- Các biến có mối quan hệ tương quan âm với biến mục tiêu (price): 1
tăng 1 giảm
+ citympg
+ highwaympg
```

```
sns.lmplot(x='enginesize', y='price', data=cars)
plt.show()
```



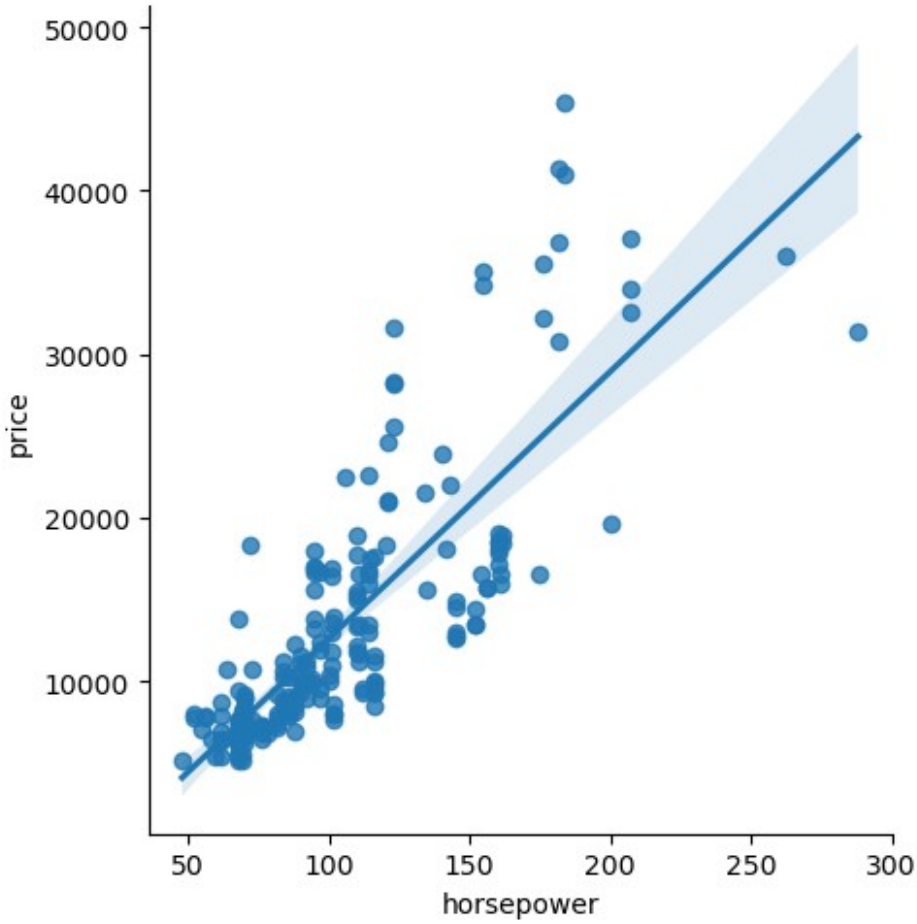

```
- Mối quan hệ tuyến tính mạnh mẽ: Khi Kích thước Động cơ tăng lên, giá
xe cũng có xu hướng tăng theo một cách khá nhất quán.
- Độ tin cậy của hồi quy: Vùng bóng mờ màu xanh nhạt bao quanh đường
hồi quy là khoảng tin cậy 95% (confidence interval). Vùng này hẹp, đặc
```

```
biệt ở khoảng giữa của dữ liệu (enginesize từ 100 đến 180), cho thấy
mô hình hồi quy dự đoán khá tốt mối quan hệ này trong phạm vi dữ liệu
đó.
- Giá trị ngoại lai: Có một số điểm dữ liệu nằm xa đường hồi quy, đặc
biệt là ở phân khúc động cơ nhỏ (ví dụ: một chiếc xe có enginesize
khoảng 90 nhưng giá trên $15,000) và phân khúc động cơ rất lớn (một số
xe có enginesize lớn nhưng giá không quá cao). Tuy nhiên, hầu hết các
điểm đều tập trung gần đường hồi quy.

sns.lmplot(x='enginesize', y='price', hue='drivewheel', data=cars,
legend=True)
plt.show()
```

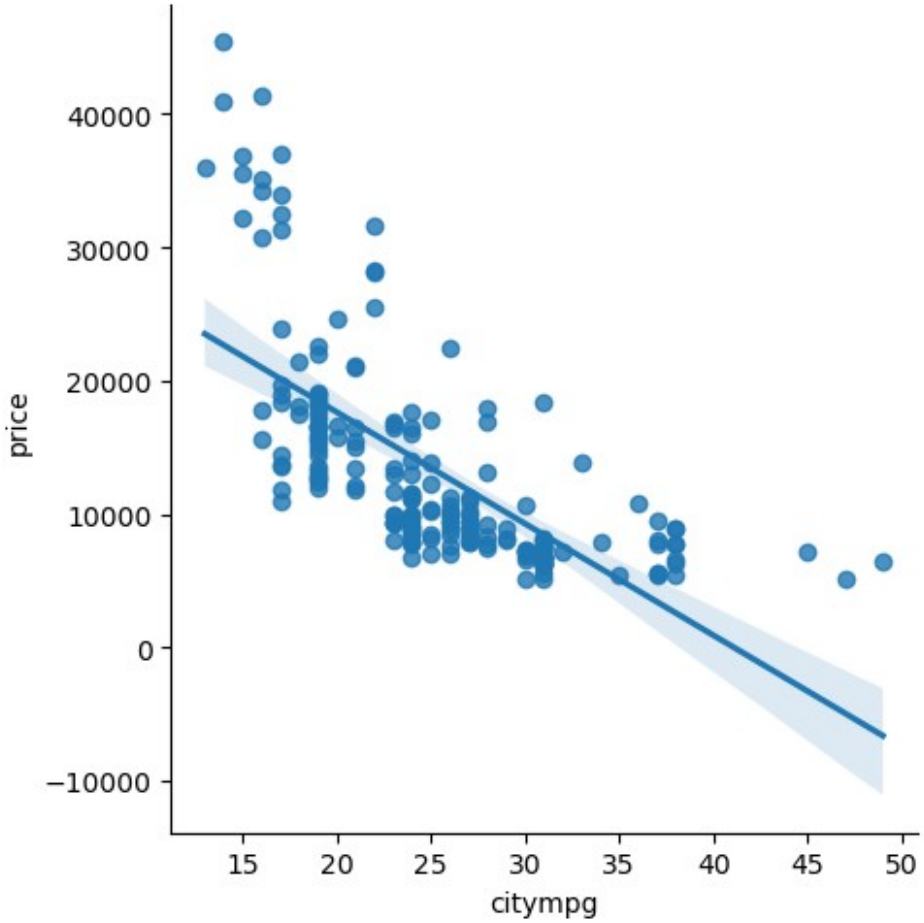



```
- Xu hướng Chung: Cả ba nhóm đều thể hiện mối quan hệ tương quan dương
mạnh giữa kích thước động cơ và giá xe.
- RWD:
+ Đây là nhóm có đường hồi quy dốc nhất. Điều này có nghĩa là trong
nhóm xe RWD, sự gia tăng kích thước động cơ dẫn đến sự tăng giá mạnh
nhất
so với các nhóm khác.
```

```
+ Các xe RWD cũng chiếm hầu hết phân khúc động cơ lớn và giá cao (phần
trên bên phải của biểu đồ).
- FWD:
+ Đây là nhóm có đường hồi quy nằm ở dưới cùng và ít dốc nhất.
+ Các xe FWD chủ yếu nằm ở phân khúc động cơ nhỏ và giá thấp (dưới
$15,000) và không có xe nào có kích thước động cơ vượt quá khoảng 160.
- 4WD:
+ Đường hồi quy nằm ở vị trí giữa FWD và RWD.
+ Tuy nhiên, dữ liệu 4WD rất ít (chỉ 9 xe), vì vậy đường hồi quy của
nó có thể không đáng tin cậy bằng hai nhóm kia, và khoảng tin cậy của
nó
(vùng bóng mờ) rộng hơn.
=> RWD là loại được sử dụng trong các mẫu xe có động cơ lớn nhất và
giá trị cao nhất, cho thấy RWD thường được sử dụng cho xe thể thao
hoặc
xe cao cấp hơn.

sns.lmplot(x='horsepower', y='price', data=cars)
plt.show()
```

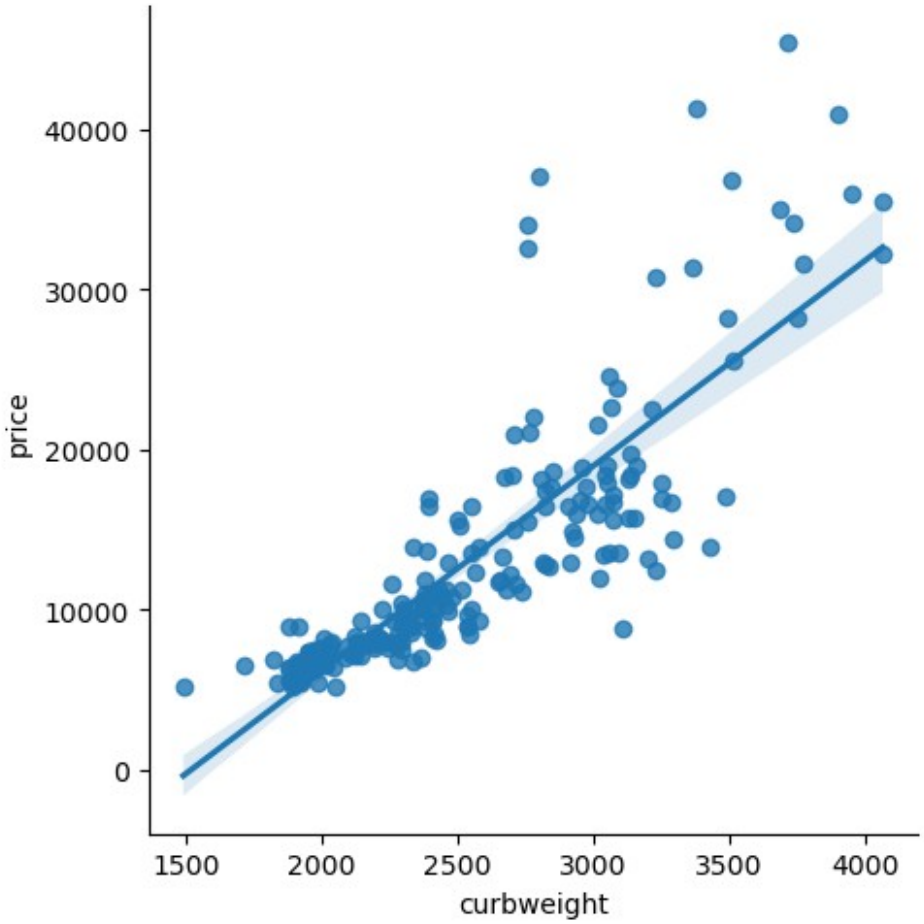


```
- Môí quan hệ tương quan dương cực mạnh: Tương tự như enginesize, mã
lực cũng có môí quan hệ tuyêń tính dương rât mạnh với giá.
- Đường hôì quy và khoảng tin cậy:
+ Đường hôì quy màu xanh đậm cho thâý xu hướng tăng giá rõ rệt theo
công suât.
+ Khoảng tin cậy (vùng bóng mờ) khá hẹp, cho thâý mô hình này có độ
chính xác cao trong việc dự đoán giá xe dựa trên mã lực.
- Phân khúc giá cao: Hâù hêt các xe có công suât trên 150 mã lực đêù
nằm trong phân khúc giá trên $25,000.
- Phân khúc mã lực thâp: Phâǹ lớn các xe có công suât dưới 100 mã lực
tập trung ở phân khúc giá thâp (dưới $15,000).
=> Mã lực là một trong những chỉ sô´mạnh nhât đểdự đoán giá trị của
một chiêć xe trong tập dữ liệu này.

sns.lmplot(x='citympg', y='price', data=cars)
plt.show()
```

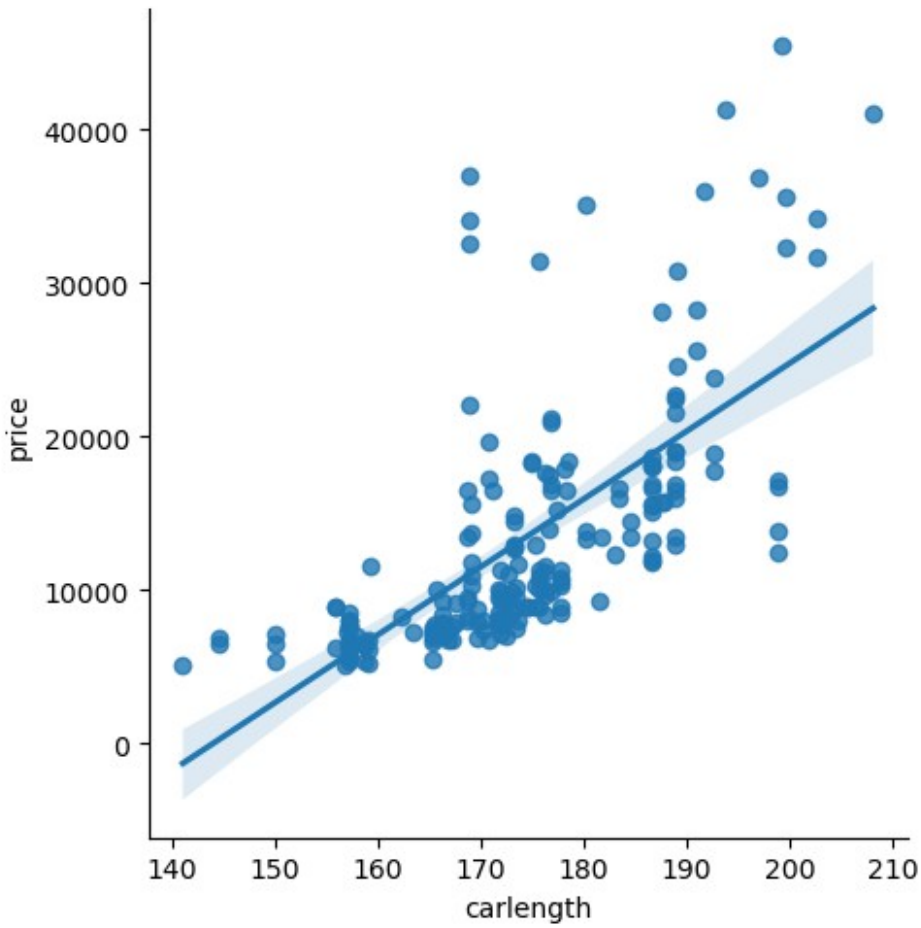



```
Chỉ sô´càng cao, xe càng tiêt kiệm nhiên liệu.

- Môí quan hệ tương quan âm mạnh: Đường hôì quy màu xanh có độ dôć âm
rõ ràng. Điêù này cho thâý môí quan hệ tương quan nghịch mạnh giữa hai
```

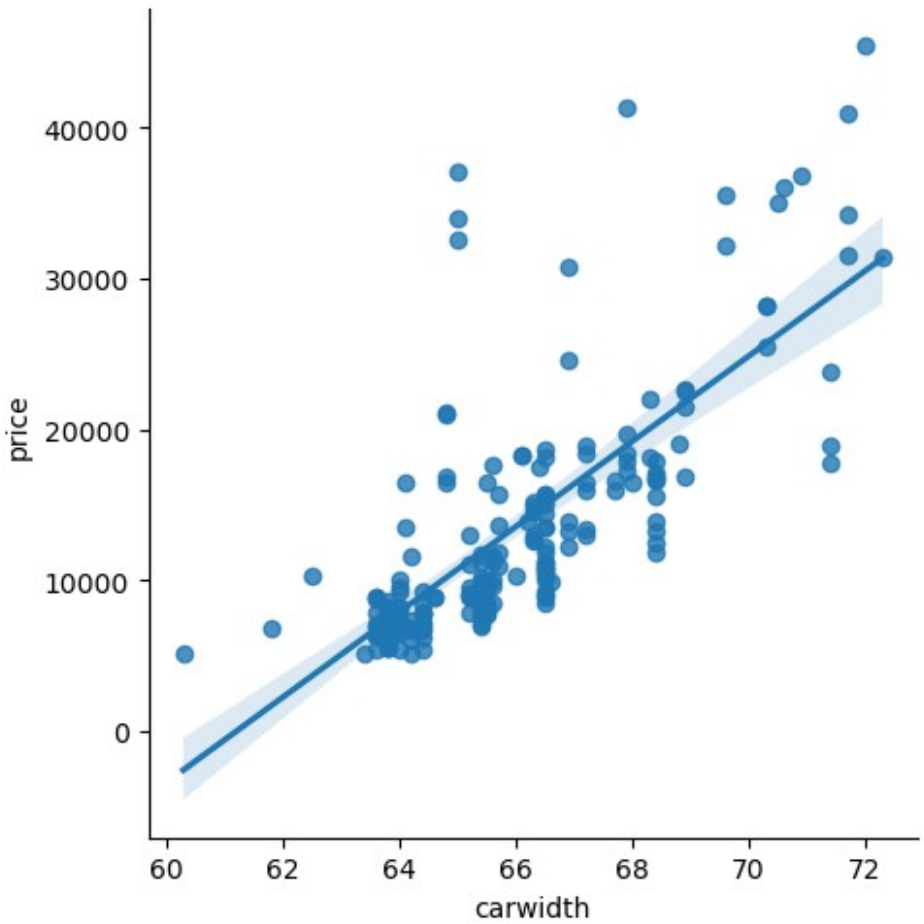
```
biến.
- Phân khúc giá cao: Hầu hết các xe có giá trên $25,000 đều có mức
tiêu thụ nhiên liệu kém (citympg dưới 25).
- Phân khúc tiết kiệm nhiên liệu: Các xe có chỉ số citympg rất cao
(trên 35) tập trung hoàn toàn ở phân khúc giá thấp (dưới $10,000).

sns.lmplot(x='curbweight', y='price', data=cars)
plt.show()
```



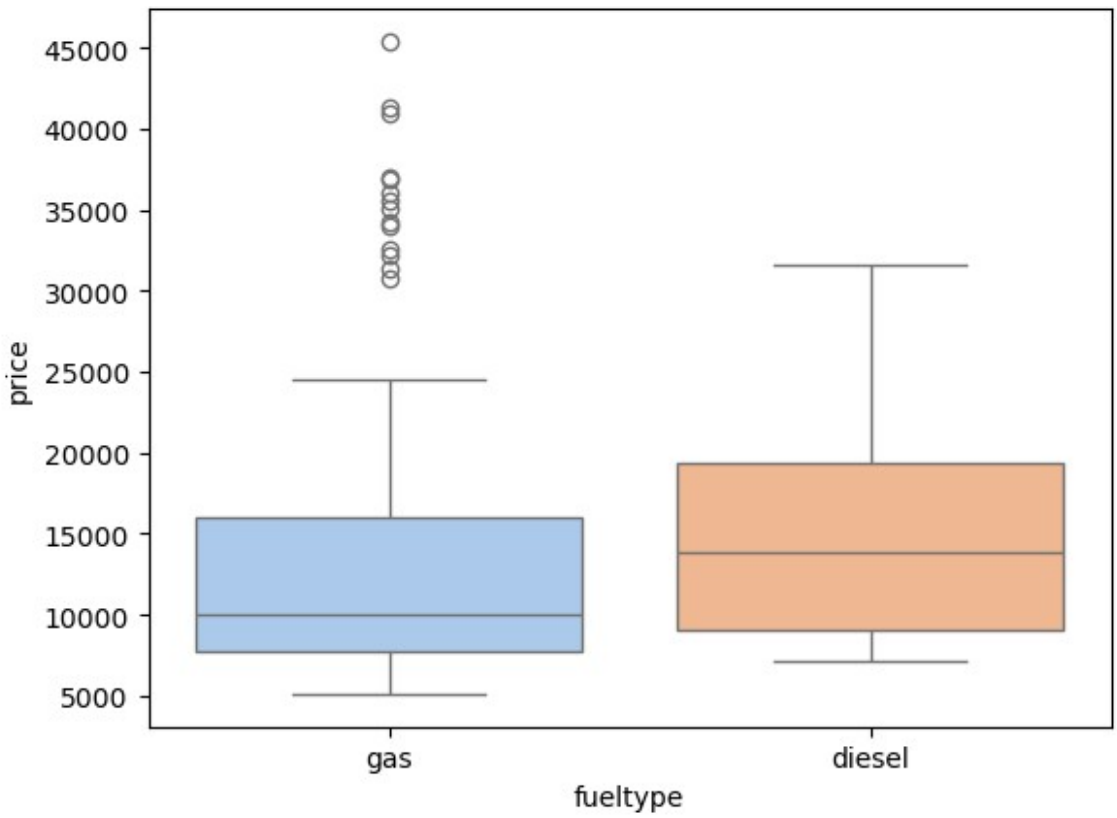

```
- Xe càng nặng (thường do vật liệu tốt hơn, động cơ lớn hơn, hoặc
nhiều tính năng hơn) thì giá càng cao.
- Xe có trọng lượng nhẹ (khoảng dưới 2500) gần như chắc chắn nằm ở
phân khúc giá thấp, trong khi xe có trọng lượng nặng (trên 3500) đều
nằm ở
phân khúc giá cao.

sns.lmplot(x='carlength', y='price', data=cars)
plt.show()
```



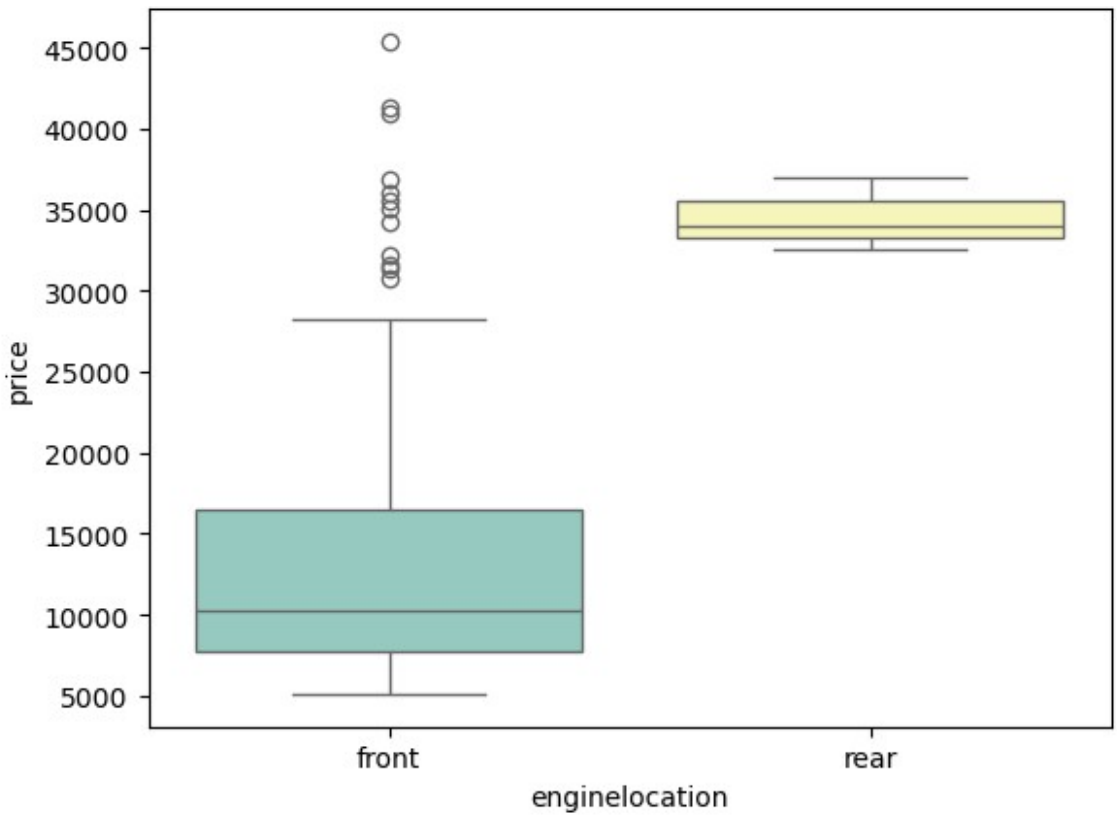

```
sns.lmplot(x='carwidth', y='price', data=cars)
plt.show()
```



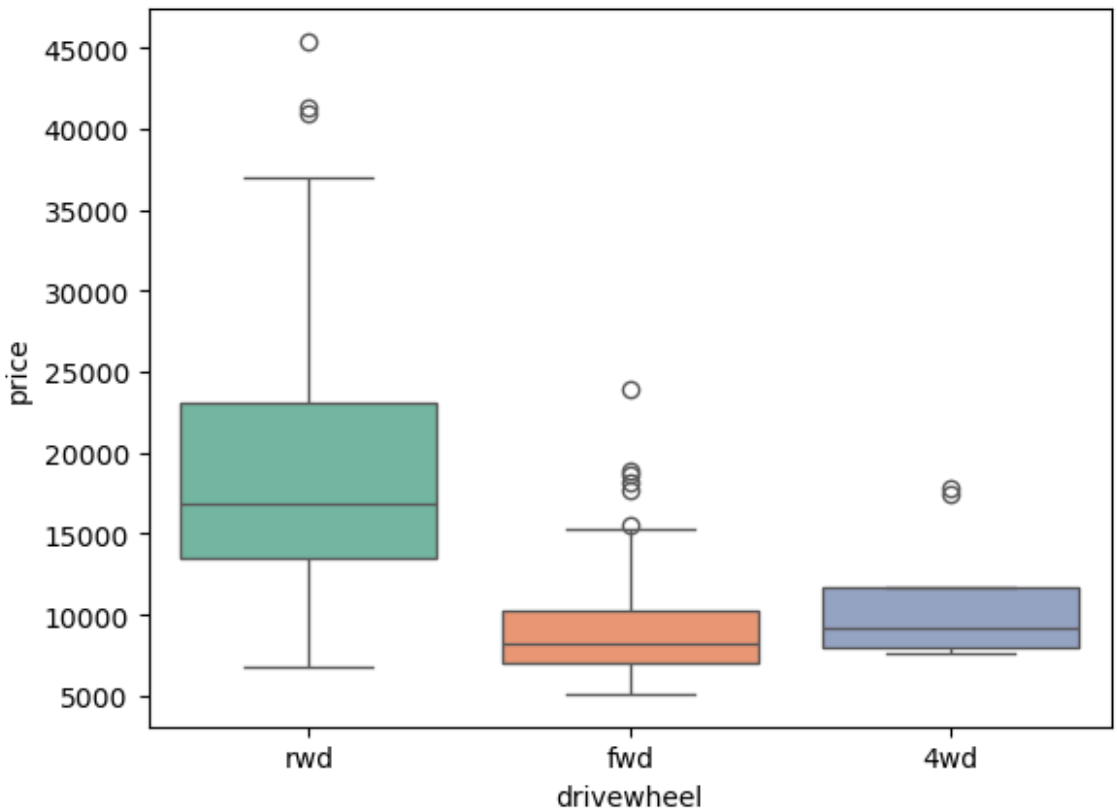

```
sns.boxplot(x='fueltype', y='price', hue='fueltype', data=cars,
palette='pastel', legend=False)
plt.show()
```



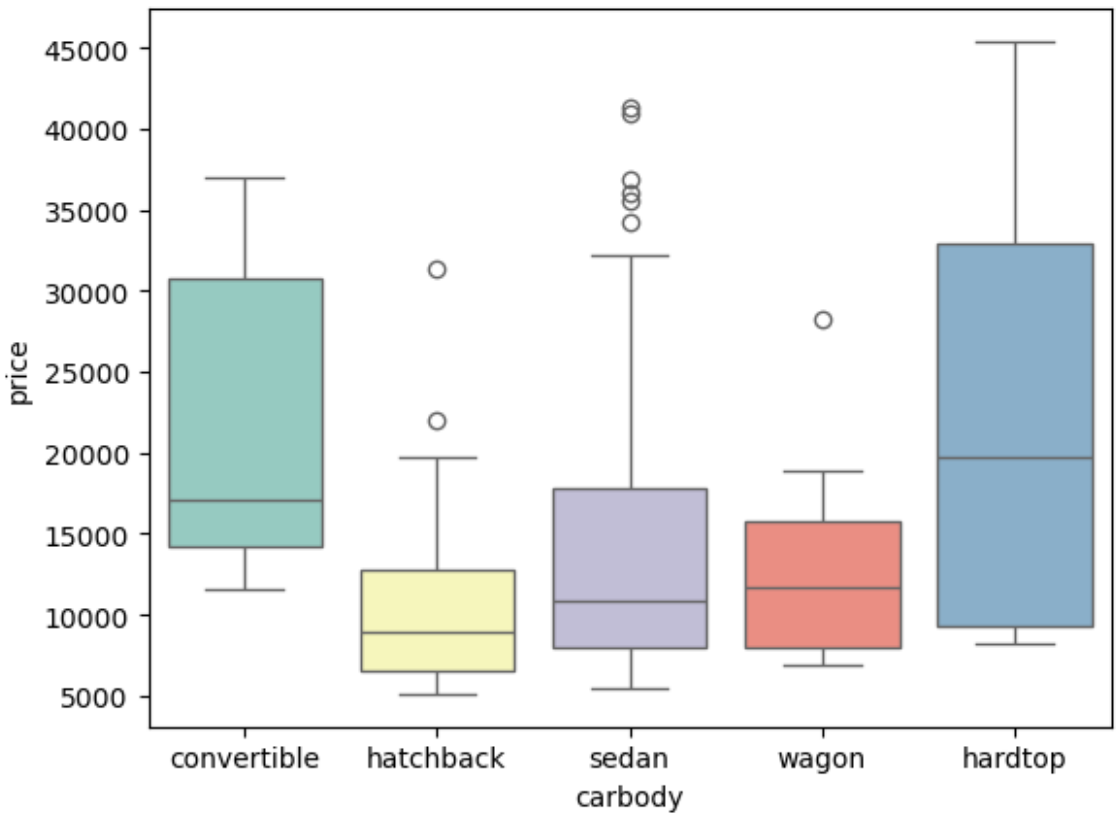

```
- Giá cao hơn: Các xe sử dụng nhiên liệu Diesel có giá trung bình và
trung vị cao hơn đáng kể so với các xe sử dụng nhiên liệu Gas.
- Giá ngoại lai: Mặc dù hầu hết các xe chạy xăng (gas) nằm ở phân khúc
giá thấp hơn, phân khúc này lại có nhiều giá trị ngoại lai (outliers)
ở mức giá rất cao (trên $40,000), điều này giải thích cho sự biến động
giá rộng hơn của xe chạy xăng.

sns.boxplot(x='enginelocation', y='price', hue='enginelocation',
data=cars, palette='Set3', legend=False)
plt.show()
```




```
- Sự khác biệt Giá cực lớn: Đây là biến phân loại có sự khác biệt về giá rõ ràng nhất trong tập dữ liệu.
- Động cơ sau đắt hơn: Xe có động cơ đặt phía sau có giá trung bình cao hơn gấp gần 3 lần so với xe có động cơ đặt phía trước.
- Động cơ sau = Xe cao cấp: Phân khúc động cơ sau (rear) chỉ có một vài điểm dữ liệu, nhưng tất cả đều nằm ở mức giá rất cao, khẳng định chúng thuộc về thị trường xe thể thao hoặc xe cao cấp.
- Động cơ trước = Phổ thông: Hầu hết các xe đều có động cơ đặt phía trước (front) và nằm trong phân khúc giá phổ thông (dưới $15,000).

sns.boxplot(x='drivewheel', y='price', hue='drivewheel', data=cars, palette='Set2', legend=False)
plt.show()
```




```
sns.boxplot(x='carbody', y='price', hue='carbody', data=cars,
palette='Set3', legend=False)
plt.show()
```




```
- Convertible (Mui trần): Có giá trung vị cao nhất (đường kẻ ngang đậm
trong hộp), cho thấy đây là phân khúc xe đắt tiền nhất. Phạm vi giá
cũng rộng.
- Hardtop (Mui cứng): Có giá trung vị cao thứ hai và cũng có một số
giá trị ngoại lai (dấu chấm) ở mức giá rất cao (trên $40,000).
- Sedan (Loại xe phổ thông): Có phân phối giá rộng nhất, từ phân khúc
thấp nhất đến cao nhất (có nhiều ngoại lai đắt tiền), cho thấy sự đa
dạng lớn trong dòng xe sedan.
- Wagon (Xe ga/Đa dụng): Có giá trung vị khá thấp và phạm vi giá tương
đối hẹp, tập trung ở phân khúc trung bình thấp.
- Hatchback (Xe hatchback): Có giá trung vị thấp nhất và phân phối giá
hẹp nhất, khẳng định đây là phân khúc xe kinh tế, giá rẻ.

=> Các biến số quan trọng nhất ảnh hưởng đến giá xe: enginesize,
horsepower, citympg, drivewheel, carbody.
```

III. MÔ HÌNH HOÁ

```
# Kiểm định giả thuyết về sự khác biệt giữa giá trung bình của
hai nhóm xe: Động cơ sau (rear) và Động cơ trước (front)
from scipy import stats

front = cars[cars['enginelocation_rear']==True]['price_capped']
rear = cars[cars['enginelocation_rear']==False]['price_capped']
```

```
levene_stat, p_levene = stats.levene(front, rear)
print("Levene p =", p_levene)
t_stat, p_value = stats.ttest_ind(front, rear, equal_var=False)
print("Welch t_stat = {:.3f}, p_value = {:.4f}".format(t_stat,
p_value))
```

```
Levene p = 0.08733517019824454
Welch t_stat = 37.472, p_value = 0.0000
```

```
/opt/anaconda3/lib/python3.13/site-packages/scipy/stats/
_axis_nan_policy.py:586: RuntimeWarning: Precision loss occurred in
moment calculation due to catastrophic cancellation. This occurs when
the data are nearly identical. Results may be unreliable.
  res = hypotest_fun_out(*samples, **kwds)
```

```
H0: Không có sự khác biệt về`giá trung bình giữa hai nhóm.
H1: Có sự khác biệt về`giá trung bình giữa hai nhóm.
Giả sử alpha = 0.05, ta thấy p_value = 0.0000 < alpha
=> Với độ sai lệch 5% thì đủ cơ sở để bác bỏ H0, nghĩa là có sự khác
biệt giữa giá trung bình của hai nhóm xe: Động cơ sau (rear)
và Động cơ trước (front)
=> Phân tích trực quan về`việc xe động cơ sau đắt hơn xe động cơ trước
đã được xác nhận bằng kiểm định thống kê.
```

```
# Kiểm định giả thuyết về sự khác biệt giữa giá trung bình của
hai nhóm xe: Động cơ gas và Động cơ diesel

gas = cars[cars['fueltype_gas']==True]['price_capped']
diesel = cars[cars['fueltype_gas']==False]['price_capped']

levene_stat, p_levene = stats.levene(gas, diesel)
print("Levene p =", p_levene)
t_stat, p_value = stats.ttest_ind(gas, diesel, equal_var=False)
print("Welch t_stat = {:.3f}, p_value = {:.4f}".format(t_stat,
p_value))
```

```
Levene p = 0.3026511603962467
Welch t_stat = -1.833, p_value = 0.0803
```

```
H0: Không có sự khác biệt về`giá trung bình giữa hai nhóm.
H1: Có sự khác biệt về`giá trung bình giữa hai nhóm.
Giả sử alpha = 0.05, ta thấy p_value = 0.0803 > alpha
=> Với độ sai lệch 5% thì không đủ cơ sở để bác bỏ H0, nghĩa là không
có sự khác biệt giữa giá trung bình của hai nhóm xe: Động cơ gas
và Động cơ diesel.
=> Sự khác biệt về`giá trung bình giữa xe chạy xăng và xe chạy diesel
là không có ý nghĩa thống kê.
```

```
# Kiểm định giả thuyết về sự khác biệt giữa giá trung bình của
hai nhóm bánh xe: RWD và FWD
```

```
rwd = cars[cars['drivewheel_rwd']==True]['price_capped']
fwd = cars[cars['drivewheel_rwd']==False]['price_capped']

levene_stat, p_levene = stats.levene(rwd, fwd)
print("Levene p =", p_levene)
t_stat, p_value = stats.ttest_ind(rwd, fwd, equal_var=False)
print("Welch t_stat = {:.3f}, p_value = {:.4f}".format(t_stat,
p_value))
```

```
Levene p = 7.101563736081212e-09
Welch t_stat = 11.054, p_value = 0.0000
```

```
H0: Không có sự khác biệt về`giá trung bình giữa hai nhóm.
H1: Có sự khác biệt về`giá trung bình giữa hai nhóm.
Giả sử alpha = 0.05, ta thấy p_value = 0.0000 < alpha
=> Với độ sai lệch 5% thì có đủ cơ sở để bác bỏ H0, nghĩa là có sự
khác biệt giữa giá trung bình của hai nhóm bánh xe: RWD và FWD.
=> Loại bánh xe ảnh hưởng đến giá xe.
```

```
# Kiểm định giả thuyết về sự khác biệt giữa giá trung bình của
hai loại hút: std và turbo

std = cars[cars['aspiration']=='std']['price']
turbo = cars[cars['aspiration']=='turbo']['price']

t_stat, p_value = stats.ttest_ind(std, turbo, equal_var=False)
print("Welch t_stat = {:.3f}, p_value = {:.4f}".format(t_stat,
p_value))
```

```
Welch t_stat = -3.108, p_value = 0.0027
```

```
H0: Không có sự khác biệt về`giá trung bình giữa hai nhóm.
H1: Có sự khác biệt về`giá trung bình giữa hai nhóm.
Giả sử alpha = 0.05, ta thấy p_value = 0.0027 < alpha
=> Với độ sai lệch 5% thì có đủ cơ sở để bác bỏ H0, nghĩa là có sự
khác biệt giữa giá trung bình của hai loại hút: std và turbo.
=> Loại hút có ảnh hưởng đến giá xe.
```

```
# Kiểm định giả thuyết về sự khác biệt giữa giá trung bình của
các nhóm kiểu thân xe

carbody_groups = cars['carbody'].unique()
price_groups = [cars[cars['carbody'] == carbody]['price'] for carbody
in carbody_groups]

# Thực hiện Kiểm định ANOVA (F-test)
# *price_groups mở danh sách các Series ra thành các đối số riêng
biệt
f_statistic, p_value = stats.f_oneway(*price_groups)
```

```
print(f"f_statistic = {f_statistic:.4f}")
print(f"p_value: {p_value:.4f}")
```

```
f_statistic = 8.0320
p_value: 0.0000
```

```
H0: Không có sự khác biệt về giá trung bình giữa các nhóm.
H1: Có sự khác biệt về giá trung bình giữa các nhóm.
Giả sử alpha = 0.05, ta thấy p_value = 0.0000 < alpha
=> Với độ sai lệch 5% thì có đủ cơ sở để bác bỏ H0, nghĩa là có sự
khác biệt giữa giá trung bình của các nhóm kiểu thân xe.
=> Kiểu dáng xe ảnh hưởng đến giá xe.
```

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

cars_processed = cars.copy()

# Chọn biến số
standard_features = [
    'horsepower_capped',
    'enginesize_capped',
    'curbweight_capped'
]
encoded_features = [
    'drivewheel_rwd', # So với 4wd (baseline)
    'drivewheel_fwd', # So với 4wd (baseline)
    'enginelocation_rear' # So với front (baseline)
]
target_feature = 'price_capped'

# Tạo X và y
X = cars_processed[standard_features + encoded_features].copy()
y = cars_processed[target_feature]

# Chia dữ liệu
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Xây dựng và Huấn luyện Mô hình Hồi quy
model = LinearRegression()
model.fit(X_train, y_train)

# Đánh giá Mô hình
r_squared_train = model.score(X_train, y_train)
r_squared_test = model.score(X_test, y_test)

# Tạo DataFrame để hiển thị các hệ số
coefficients_cars = pd.DataFrame({
    'Variable': X.columns,
```

```
    'Coefficient': model.coef_
}).sort_values(by='Coefficient', ascending=False)


print("--- Kết quả Mô hình ---")
print("Hệ số hồi quy (Coefficients):")
print(coefficients_cars.to_markdown(numalign="left", stralign="left",
floatfmt=".2f"))
print("\nĐộ chính xác R-squared (Tập huấn luyện):
{:.4f}".format(r_squared_train))
print("Độ chính xác R-squared (Tập kiểm tra):
{:.4f}".format(r_squared_test))
```

```
--- Kết quả Mô hình ---
Hệ số hồi quy (Coefficients):
|    | Variable            | Coefficient   |
|:---|:--------------------|:--------------|
| 5  | enginelocation_rear | 10159.16      |
| 3  | drivewheel_rwd      | 2113.84       |
| 4  | drivewheel_fwd      | 1338.37       |
| 0  | horsepower_capped   | 41.00         |
| 1  | enginesize_capped   | 19.91         |
| 2  | curbweight_capped   | 7.06          |

Độ chính xác R-squared (Tập huấn luyện): 0.8481
Độ chính xác R-squared (Tập kiểm tra): 0.8414

Coefficients cho biết mức độ ảnh hưởng của mỗi biến đối với giá xe
(USD)
+ enginelocation_rear (Tác động lớn nhất): Nếu động cơ đặt phía sau
(so với phía trước), giá xe tăng thêm $10159.16
+ drivewheel_rwd: Xe RWD có giá cao hơn $2113.84 so với xe 4WD (biến
cơ sở)
+ drivewheel_fwd: Xe FWD có giá cao hơn $1338.37 so với xe 4WD (biến
cơ sở)
+ enginesize_capped: Khi kích thước động cơ tăng thêm 1 đơn vị, giá xe
tăng thêm $19.91
+ horsepower_capped: Khi công suất tăng thêm 1 mã lực, giá xe tăng
thêm $41.00
+ curbweight_capped: Khi trọng lượng xe tăng thêm 1 pound, giá xe tăng
thêm $7.06

R-squared (Tập kiểm tra): 0.8414
=> 84.14% sự biến thiên của giá xe có thể được giải thích bằng mô hình
này và các biến số đã chọn.
Đây là một mô hình dự đoán giá xe đáng tin cậy.
```

```
# Phương trình hồi quy tuyến tính
print(f'''The linear model is:
Y = {model.intercept_}
```

```
+ {model.coef_[0]}*X1
+ {model.coef_[1]}*X2
+ {model.coef_[2]}*X3
+ {model.coef_[3]}*X4
+ {model.coef_[4]}*X5
+ {model.coef_[5]}*X6
''')
```

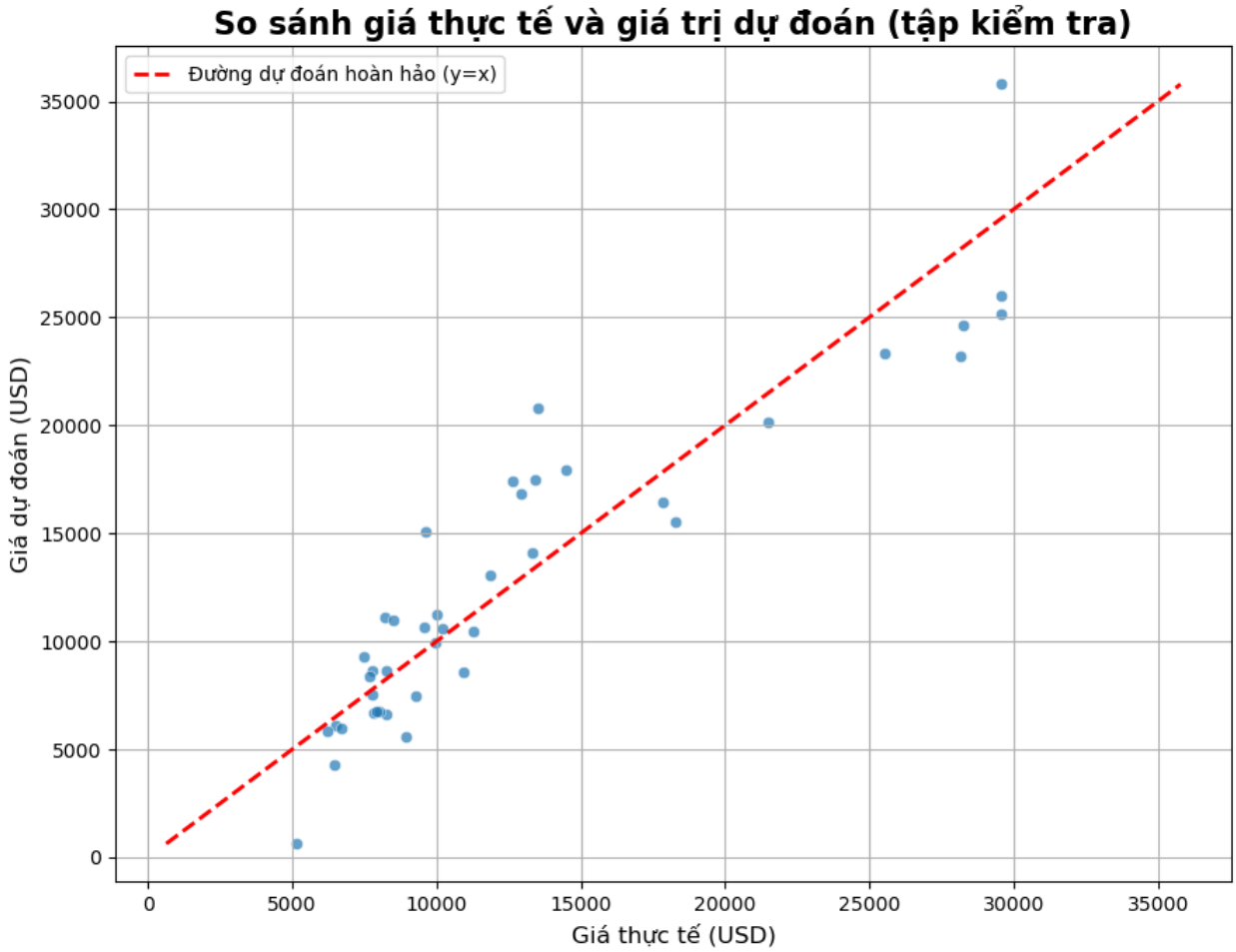
```
The linear model is:
Y = 11033.382446948535
+ 1455.2042673434887*X1
+ 674.8766371814504*X2
+ 3665.7831881978786*X3
+ 2113.8423314803945*X4
+ 1338.365224715837*X5
+ 10159.16424589683*X6
```

```
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
print(rmse)
```

```
2929.3261898945084
```

=> Chỉ số RMSE cho biết mô hình dự đoán giá xe bị sai lệch trung bình khoảng $2929.33 so với giá thực tế.

```
# Tạo Biểu đồ phân tán
plt.figure(figsize=(9, 7))
sns.scatterplot(x=y_test, y=y_pred, alpha=0.7)

# Vẽ đường dự đoán hoàn hảo (y=x). Đường này biểu thị nơi Giá Dự đoán = Giá Thực tế
max_val = max(y_test.max(), y_pred.max())
min_val = min(y_test.min(), y_pred.min())
plt.plot([min_val, max_val], [min_val, max_val], color='red', linestyle='--', linewidth=2, label='Đường dự đoán hoàn hảo (y=x)')

plt.title('So sánh giá thực tế và giá trị dự đoán (tập kiểm tra)', fontsize=16, fontweight='bold')
plt.xlabel('Giá thực tế (USD)', fontsize=12)
plt.ylabel('Giá dự đoán (USD)', fontsize=12)
plt.legend()
plt.grid(True)
plt.tight_layout()
```



- Hiệu suất Tổng thể: Hầu hết các điểm dữ liệu đều tập trung rất gần đường dự đoán hoàn hảo (y=x), đặc biệt là trong phạm vi giá dưới $20,000 USD. Điều này xác nhận chỉ số R^2 cao (0.8414) và cho thấy mô hình hoạt động tốt đối với phần lớn các mẫu xe.
- Sai số:
+ Giá trị ngoại lai: Có một vài điểm dữ liệu nằm xa đường y=x, chủ yếu ở phân khúc giá cao (trên $30,000) và các điểm giá thấp.
+ Giá cao: Đối với một số xe có giá thực tế rất cao, mô hình có xu hướng dự đoán thấp hơn giá trị thực tế.
- Tính tuyến tính: Mối quan hệ giữa giá thực tế và giá dự đoán là tuyến tính và mạnh mẽ.